

OPENSIFT CLUSTER

A to Z

A Complete Beginner-to-Advanced Guide — Concepts, Architecture, Acronyms, Real-World Scenarios, and a Hands-On Homelab Build on VMware ESXi

Prepared for: Mohammed Shafeeulla (Md)

Homelab: VMware ESXi · 32 GB RAM · 320 GB SSD · VS Code

AI BlockMaker — Big ideas. Broken into blocks.

Table of Contents

Chapter 1 — Why This Book Exists, and Why You Are Learning This	10
1.1 Who this is written for	10
1.2 The problem this technology solves	10
1.3 Why you, specifically, are learning this	10
1.4 What "learning it properly" looks like	11
Chapter 2 — Containers Before Kubernetes: The Building Blocks	12
2.1 What a container actually is	12
2.2 Images, layers, and registries	12
2.3 Docker vs Podman vs CRI-O — three names you will keep seeing	12
2.4 Why one container is not enough — the case for orchestration	13
Chapter 3 — Kubernetes Fundamentals (the Engine Under OpenShift)	14
3.1 The cluster and its two kinds of machines	14
3.2 The core control plane components	14
3.3 The core worker-node components	14
3.4 The core Kubernetes objects, explained with "why"	15
3.5 The reconciliation loop — the single idea that explains everything	16
Chapter 4 — What OpenShift Adds on Top of Kubernetes	17
4.1 Projects (OpenShift's Namespace, with extras)	17
4.2 Security Context Constraints (SCC) — OpenShift's stricter default security	17
4.3 Routes — OpenShift's original way to expose applications	17
4.4 Built-in Integrated Image Registry	17
4.5 Source-to-Image (S2I) and BuildConfig — build without writing a Dockerfile	18
4.6 DeploymentConfig vs Deployment	18
4.7 The Web Console	18
4.8 OAuth-based Authentication and Identity Providers	18
4.9 Operators and the Operator Lifecycle Manager (OLM)	19
4.10 Summary table — Kubernetes vs OpenShift	19
Chapter 5 — OpenShift Architecture, End to End	20
5.1 The three node roles	20
5.2 RHCOS — the operating system every node runs	20
5.3 Operators all the way down — the Cluster Version Operator	20

5.4 The request path — from a browser to your Pod, step by step	21
5.5 How a Pod actually gets scheduled and started	21
5.6 Multi-cluster / hybrid concepts you will hear about	21
Chapter 6 — Acronym Glossary (Read Once, Reference Forever)	22
Chapter 7 — Networking in OpenShift	27
7.1 Why cluster networking is hard in the first place	27
7.2 OVN-Kubernetes — the default CNI plugin	27
7.3 The four networking problems Kubernetes/OpenShift networking must solve	27
7.4 Services — the four types	27
7.5 Routes in depth	27
7.6 Ingress vs Route	27
7.7 NetworkPolicy	28
Chapter 8 — Security: RBAC, SCC, Projects, and Secrets	29
8.1 Why security is a first-class chapter	29
8.2 Authentication vs Authorization	29
8.3 RBAC building blocks	29
8.4 SCC in more depth	29
8.5 Secrets and their real-world limitations	29
8.6 Image security	29
Chapter 9 — The Application Lifecycle: From Code to Running Pod	30
9.1 Three ways to get an application running	30
9.2 Templates vs Helm	30
9.3 A full walk-through	30
9.4 Rolling updates, readiness, and liveness	30
9.5 Rollbacks	30
Chapter 10 — Operators and the Operator Lifecycle Manager (OLM)	31
10.1 The problem Operators solve	31
10.2 Custom Resource Definitions (CRDs)	31
10.3 OperatorHub and OLM in practice	31
10.4 Why this matters for your AI/ML background	31
Chapter 11 — CI/CD on OpenShift	32
11.1 Why CI/CD and OpenShift go together	32
11.2 OpenShift Pipelines (built on Tekton)	32

11.3 GitOps with ArgoCD (OpenShift GitOps)	32
11.4 Jenkins	32
Chapter 12 — Monitoring and Logging	33
12.1 Monitoring stack	33
12.2 Logging: from EFK toward Loki	33
12.3 The three pillars of observability	33
Chapter 13 — Storage: PV, PVC, StorageClass, and CSI	34
13.1 Why Pods need special help with storage	34
13.2 The core objects	34
13.3 Access modes	34
13.4 vSphere-specific storage in OpenShift	34
13.5 Local storage for small/home setups	35
Chapter 14 — Installation Methods: The Big Picture Before You Install Anything	36
14.1 The two fundamental installation philosophies	36
14.2 Full multi-node OpenShift vs the lighter-weight alternatives	36
14.3 Why the free/community route makes sense for you specifically	37
Chapter 15 — Planning Your Cluster on Your Actual ESXi Box	38
15.1 Doing the resource math honestly	38
15.2 The recommended path, in order	38
15.3 Network planning on ESXi	39
15.4 Disk planning across 320 GB	39
Chapter 16 — Step-by-Step: Installing on Your ESXi Homelab	40
16.1 Path A — CRC (do this first, today)	40
16.2 Path B — Single Node OKD on ESXi (the "real install" milestone)	40
16.3 Common install-time pitfalls, specific to your setup	41
Chapter 17 — Using VS Code as Your OpenShift Workstation	42
17.1 Why VS Code fits this workflow well	42
17.2 The extensions worth installing	42
17.3 A practical day-to-day loop	42
17.4 Keeping kubeconfig organised	42
Chapter 18 — Day-2 Operations: Running the Cluster, Not Just Installing It	43
18.1 Why "Day 2" is its own category	43
18.2 Upgrades	43

18.3 Scaling	43
18.4 Backup and disaster recovery	43
18.5 Node lifecycle: cordon, drain, and maintenance	43
18.6 Certificate rotation	43
Chapter 19 — Real-World Use Cases and Industry Scenarios	45
19.1 Banking and financial services	45
19.2 Telecommunications (telco) and edge computing	45
19.3 Retail	45
19.4 Healthcare	45
19.5 Hybrid and multi-cloud strategy	45
19.6 AI/ML workloads (directly relevant to you)	46
19.7 A composite "day in the life" scenario, tying concepts together	46
Chapter 20 — Common Troubleshooting Scenarios	47
20.1 Pod stuck in "Pending"	47
20.2 Pod stuck in "CrashLoopBackOff"	47
20.3 PVC stuck in "Pending"	47
20.4 Node shows "NotReady"	47
20.5 Route/application unreachable from a browser, but the Pod is Running	47
20.6 Build failing (S2I / BuildConfig)	47
20.7 Cluster upgrade or ClusterOperator stuck "Degraded"	48
Chapter 21 — Interview Preparation: Questions and Strong Answers	49
21.1 Conceptual questions	49
21.2 Scenario-based questions	49
21.3 More scenario and architecture questions	50
21.4 A note on how to actually answer in an interview	51
Chapter 22 — Where to Go From Here	52
22.1 A realistic 90-day learning path building on this book	52
22.2 Certifications worth knowing about	52
22.3 Content creation angle for AI BlockMaker	52
Chapter 23 — Annotated YAML: Reading and Writing Manifests Fluently	53
23.1 Why YAML, and why you must be able to read it without a console	53
23.2 The universal shape every object shares	53
23.3 A Service manifest, annotated	54

23.4 A Route manifest, annotated	54
23.5 A PersistentVolumeClaim manifest, annotated	55
23.6 A ConfigMap and a Secret, side by side	55
23.7 Wiring a ConfigMap and Secret into a Pod	56
Chapter 24 — OpenShift Virtualization: Running VMs Alongside Containers	57
24.1 Why this chapter matters specifically to you	57
24.2 How it actually works under the hood	57
24.3 Why an organisation would want this instead of just keeping VMware	57
24.4 A minimal VM manifest, for orientation	57
Chapter 25 — Service Mesh: When Applications Need to Talk to Each Other Intelligently	59
25.1 The problem that appears once you have many microservices	59
25.2 How a service mesh works: the sidecar pattern	59
25.3 Concrete capabilities and why they matter	59
25.4 When you actually need one	59
Chapter 26 — Multi-Tenancy, Governance, and Cost Awareness	61
26.1 What "multi-tenancy" means in this context	61
26.2 ResourceQuota and LimitRange — stopping one tenant from starving everyone else	61
26.3 Why this matters even in a homelab of one	61
26.4 Cost awareness on a homelab specifically	61
26.5 Chargeback and showback in real organisations	62
Chapter 27 — How OpenShift Compares to Other Platforms	63
27.1 Why this comparison matters	63
27.2 Managed cloud Kubernetes: EKS, AKS, GKE	63
27.3 Rancher	63
27.4 Plain/vanilla Kubernetes (self-assembled)	63
27.5 A grounded summary table	63
27.6 The honest answer to "which one is best"	64
Chapter 28 — A Complete End-to-End Worked Project	65
28.1 The goal of this chapter	65
28.2 The scenario	65
28.3 Step 1 — create an isolated Project with a quota	65
28.4 Step 2 — install PostgreSQL via an Operator	65

28.5 Step 3 — build and deploy the front end from source	66
28.6 Step 4 — restrict network access with a NetworkPolicy	66
28.7 Step 5 — add a HorizontalPodAutoscaler	66
28.8 Step 9 — verify the whole system end to end	67
28.9 What this project teaches, chapter by chapter	67
Chapter 29 — Frequently Asked Beginner Questions	68
29.1 "Do I need to know Docker before learning OpenShift?"	68
29.2 "Is OpenShift the same as Kubernetes, just renamed?"	68
29.3 "Why does my container fail with a permissions error that never happened on Docker Desktop?" ⁶⁸	
29.4 "What is the actual difference between a Project and a Namespace?"	68
29.5 "Can I just install Docker Desktop's Kubernetes and call it close enough to OpenShift?"	68
29.6 "Is my 32 GB homelab actually 'real world' experience, or just a toy?"	68
29.7 "How long does it realistically take to become 'good' at this?"	69
29.8 "What should I actually build first, this week?"	69
Chapter 30 — Preparing ESXi Itself, Click by Click	70
30.1 Why this chapter exists separately from Chapter 16	70
30.2 Confirm virtualization prerequisites on the ESXi host itself	70
30.3 Creating the VM for a Single Node OKD build	70
30.4 A pre-flight checklist worth running every time	71
30.5 What "healthy" looks like immediately after first boot	71
Chapter 31 — Object Kind Quick Reference	72
31.1 Why a table like this earns its place	72
Chapter 32 — How Your RHCSA Study Connects Directly to OpenShift	75
32.1 Why this bridge matters	75
32.2 Storage and LVM → PersistentVolumes	75
32.3 Boot process → understanding node startup and RHCOS	75
32.4 Recovery techniques → cluster-level disaster recovery instincts	75
32.5 SELinux and firewalld → SCC and NetworkPolicy	75
32.6 The honest takeaway	76
Chapter 33 — Self-Test: Check Your Understanding	77
33.1 How to use this chapter	77
33.2 Foundational questions	77
33.3 OpenShift-specific questions	77

33.4 Homelab and infrastructure questions	77
33.5 Troubleshooting scenarios	78
33.6 Answer-writing exercise	78

Appendix — Command Cheat Sheet 79

A.1 Cluster and login	79
A.2 Deploying and inspecting applications	79
A.3 Scaling and rollout	79
A.4 Storage	79
A.5 Security and access	79
A.6 Day-2 / node operations	79
A.7 Troubleshooting	80

Chapter 1 — Why This Book Exists, and Why You Are Learning This

1.1 Who this is written for

You are starting from zero on OpenShift, but you are not starting from zero on computers. You already run a homelab: a VMware ESXi host with 32 GB of RAM and a 320 GB SSD, and you edit everything from VS Code on your laptop. This book assumes exactly that starting point. Every acronym is expanded the first time it is used. Every concept is explained with "why do we even need this" before "how do we configure this." By the last chapter you will have a working OpenShift (or OKD) environment running on your own ESXi box, understand every layer underneath it, and know how to talk about it in an interview or explain it on your YouTube channel, AI BlockMaker.

1.2 The problem this technology solves

Before containers, a company that built a piece of software had to also ship, by hand, a description of exactly which operating system, which libraries, and which configuration the software needed to run. "It works on my machine" was the most common sentence in the software industry, and it was a symptom of a real problem: the environment where code was written was never quite the same as the environment where it was run in production.

Containers solved the "it works on my machine" problem by packaging the application together with everything it needs — libraries, runtime, configuration files — into one unit that behaves the same everywhere. Docker made this mainstream around 2013. But a single container is not a system. Real applications are made of many containers working together: a web front end, an API, a database cache, a message queue. Someone has to decide which physical or virtual machine each container runs on, restart it if it crashes, give it network access to its neighbours, scale it up when traffic increases, and roll out new versions without downtime. Doing all of that by hand for hundreds of containers is not humanly possible at any real scale. That coordination problem is called container orchestration, and the tool that won that space is Kubernetes.

Kubernetes itself is powerful but intentionally bare. It is a set of APIs and a scheduler — it does not ship a build system, a container registry, a user-friendly console, an authentication system, or default security rules out of the box. Every company that adopts raw Kubernetes ends up bolting on ten different tools to get those things, and every company bolts on a slightly different set, which becomes very hard to support. Red Hat's OpenShift is Kubernetes plus all of those missing pieces, pre-integrated, tested together, and supported as one product. That is the one-sentence definition you should keep in your head for the rest of this book:

OpenShift = Kubernetes + a built-in container build system + a built-in image registry + a web console + enterprise authentication + stricter default security + a curated marketplace of ready-to-run software (Operators) — all packaged, tested, and supported as a single product by Red Hat.

1.3 Why you, specifically, are learning this

You come from a Machine Learning / AI engineering background and you are simultaneously building Linux and DevOps skills through your Prodevans internship. This combination is not a coincidence you should worry about — it is precisely the profile that is in demand right now. Companies that run AI workloads need people who understand both the model and the infrastructure that serves it reliably, scales it, and secures it. OpenShift, in particular, has a first-class story for exactly this: OpenShift AI (previously called Open Data Hub / RHOAI) is Red Hat's platform for running Jupyter notebooks, model training, and model serving on top of the same OpenShift cluster you are about to build. Learning OpenShift from the ground up therefore feeds directly into both your infrastructure career path and your AI career path.

1.4 What "learning it properly" looks like

There are three depths at which people "know" OpenShift, and this book takes you through all three, in order:

- 1 **Conceptual depth** — you can explain, in plain words, what a Pod is, why we need a Service, what a Route does, and why the whole system is arranged the way it is. This is what lets you pass an interview and mentor others.
- 2 **Operational depth** — you can install a cluster, deploy an application, expose it to the internet, scale it, look at its logs, and fix it when something breaks. This is what lets you do the job day to day.
- 3 **Architectural depth** — you understand what happens underneath: how the API server talks to etcd, how the scheduler picks a node, how a container image becomes a running process, how network packets travel from the internet to your Pod. This is what separates someone who follows a tutorial from someone who can debug a 2 a.m. production incident.

By design, this book front-loads concepts, builds architectural understanding chapter by chapter, and lands on your specific ESXi hardware with a realistic, tested installation plan — because 32 GB of RAM and 320 GB of disk is a real constraint, and pretending otherwise would set you up to fail on installation day.

Chapter 2 — Containers Before Kubernetes: The Building Blocks

2.1 What a container actually is

A container is not a small virtual machine, even though it looks like one from the outside. A virtual machine (VM) virtualizes hardware — it runs its own full operating system kernel on top of a hypervisor, which is why a VM can take a minute to boot and why it needs gigabytes just for the OS. A container instead virtualizes the operating system: many containers on one host share the same Linux kernel, and are isolated from each other using two Linux kernel features:

- **Namespaces** — give each container its own private view of process IDs, network interfaces, mount points, and hostname, so a process inside the container thinks it is the only thing running on the machine.

- **Control groups (cgroups)** — let the kernel limit and account for how much CPU, memory, and I/O each container is allowed to use, so one noisy container cannot starve the others.

Because a container does not boot its own kernel, it starts in milliseconds instead of minutes, and a single container image is typically tens or a few hundred megabytes instead of the multiple gigabytes a full VM image needs. This is why the industry moved so quickly from VM-per-application to container-per-application: far higher density on the same hardware, and far faster start-up.

2.2 Images, layers, and registries

A **container image** is a read-only, versioned snapshot of a filesystem plus metadata describing how to run it (the entrypoint command, exposed ports, environment variables). Images are built in layers — each instruction in a build file adds one layer, and layers are cached and shared between images, which is why pulling a second image that shares a base with an image you already have is fast.

A **container registry** is simply a server that stores and serves these images by name and tag, the same way a Git remote stores commits. Docker Hub is the best-known public registry; quay.io (also a Red Hat product) is another very common one; and — importantly for this book — every OpenShift cluster ships its own **internal image registry** built in, so you rarely need to reach out to the public internet for images your own team builds.

2.3 Docker vs Podman vs CRI-O — three names you will keep seeing

- **Docker** is the tool that popularized containers. It includes a daemon (a background service running as root), a CLI, and its own image format, which later became the industry-standard OCI (Open Container Initiative) format.

Podman is Red Hat's alternative to Docker. It is command-line compatible (`podman run` works almost exactly like `docker run`) but has no long-running root daemon — each container is a direct child process of the user who started it, which is more secure by default. This is the tool you will actually use day to day on RHEL (Red Hat Enterprise Linux) and on your homelab, since Red Hat has stopped shipping Docker on RHEL in favour of Podman.

CRI-O is a lightweight container runtime built specifically to implement Kubernetes' Container Runtime Interface (CRI) — nothing more, nothing less. OpenShift uses CRI-O internally on every node to actually start and stop containers. You will not usually run `crio` commands yourself; the platform calls it on your behalf.

Why this matters: when someone says "OpenShift doesn't use Docker," they mean OpenShift's nodes use CRI-O as the low-level engine, not the Docker daemon. You can still build images the "Docker way," using a Dockerfile, and it will work fine — the build tool and the run-time engine are different things.

2.4 Why one container is not enough — the case for orchestration

Imagine you containerize a web application. In production you need at least: multiple copies of it for reliability (if one crashes, others keep serving traffic), a way to route traffic only to the healthy copies, a way to roll out version 2.0 without downtime, a way to automatically restart a crashed copy, a way to store secrets like database passwords outside the image, and a way to attach persistent disk to a database container so data survives a restart. None of this is provided by the container engine itself — Podman or Docker only know how to run one container at a time on one machine. The tool that manages all of the above, across a whole fleet of machines, is a **container orchestrator**, and Kubernetes is the one the industry standardized on.

Chapter 3 — Kubernetes Fundamentals (the Engine Under OpenShift)

OpenShift is, under the hood, a Kubernetes distribution. Every OpenShift-specific idea in later chapters builds on the objects described here, so this chapter is worth reading slowly even if you have seen some of these words before.

3.1 The cluster and its two kinds of machines

A **Kubernetes cluster** is a set of machines (physical or virtual) working together, split into two roles:

- Control plane nodes** (sometimes still called "masters") — run the brain of the cluster: the API, the scheduler, the components that keep track of desired vs actual state. In OpenShift these are literally called **Control Plane nodes**.

- Worker nodes** (sometimes called "compute nodes") — run your actual application containers.

A production cluster usually has 3 control plane nodes (for high availability — 2 can fail and the third keeps quorum) and as many worker nodes as capacity requires. A learning/home lab cluster can legitimately be a single node playing both roles — you'll see this option, called Single Node OpenShift (SNO), in Chapter 13, and it is very relevant to your 32 GB box.

3.2 The core control plane components

- kube-apiserver (API server)** — the front door to the entire cluster. Every single action — whether from the `kubectl` command line, the web console, or an internal controller — goes through the API server, which validates it and stores it. Nothing in Kubernetes talks to anything else directly; everything goes through this API.

- etcd** — a distributed, consistent key-value database that is the single source of truth for the entire cluster's state: every object, every configuration, every secret is stored here. If etcd is lost with no backup, the cluster's state is lost. This is why etcd backup is one of the most important "Day-2" operations you will learn later.

- kube-scheduler** — watches for newly created Pods that have not yet been assigned to a node, and decides which node they should run on, based on requested CPU/memory, node capacity, affinity rules, and taints/tolerations (explained in Chapter 4).

- kube-controller-manager** — runs a set of control loops, each watching one type of resource and constantly working to make the actual state match the desired state (for example: "the Deployment says 3 replicas should exist, only 2 do, create one more").

- cloud-controller-manager** — a control loop that talks to the underlying infrastructure provider (AWS, vSphere, bare metal) for things like provisioning load balancers or storage volumes. On a vSphere/ESXi install, this is what lets OpenShift ask vCenter to create virtual disks or VMs automatically.

3.3 The core worker-node components

- bullet**kubenet** — the agent that runs on every node (control plane nodes run one too) and is responsible for making sure the containers described for that node are actually running and healthy. It talks to the container runtime to start/stop containers and reports status back to the API server.
- bullet**kube-proxy** — maintains network rules on each node so that traffic sent to a Service's virtual IP gets routed to one of the actual Pods behind it.
- bullet**Container runtime** — CRI-O in OpenShift's case, as covered in Chapter 2.

3.4 The core Kubernetes objects, explained with "why"

- bullet**Pod** — the smallest deployable unit. A Pod is one or more containers that always run together on the same node and share networking (same IP address) and optionally storage. Most Pods have exactly one container; multiple containers in one Pod ("sidecar" pattern) are used when a helper process must live tightly coupled to the main one (e.g., a log-shipper sitting next to a web server). **Why it exists:** Kubernetes needed an atomic scheduling unit bigger than "one process" so that tightly-coupled helper containers can be guaranteed to land together.
- bullet**ReplicaSet** — ensures a specified number of identical Pod copies are running at all times. **Why:** if a single Pod is the unit, something has to keep N of them alive.
- bullet**Deployment** — a higher-level object that manages ReplicaSets for you and provides rolling updates and rollbacks. In practice, you almost never create a ReplicaSet directly — you create a Deployment, and it creates and manages ReplicaSets underneath. **Why:** you want to change your application's image version without downtime, and be able to undo a bad rollout in one command.
- bullet**Service** — a stable, unchanging virtual IP address and DNS name that always points at a healthy set of Pods, even as individual Pods are replaced, rescheduled, or scaled. **Why:** Pods are disposable and get new IP addresses every time they restart; something in front of them needs to stay constant so other parts of the system can find them reliably.
- bullet**Namespace** — a way to divide one physical cluster into multiple virtual clusters for organizational and access-control purposes (e.g., `dev`, `test`, `prod`, or one namespace per team). In OpenShift, a namespace with extra metadata is called a **Project** (Chapter 4).
- bullet**ConfigMap** — stores non-sensitive configuration data (e.g., a feature flag, a URL) outside of the container image, so the same image can be reused across environments with different configuration.
- bullet**Secret** — like a ConfigMap but for sensitive values (passwords, API keys, TLS certificates), base64-encoded at rest and access-controlled separately.
- bullet**Volume / PersistentVolume (PV) / PersistentVolumeClaim (PVC)** — mechanisms for giving a Pod disk storage that can (with PV/PVC) survive the Pod being deleted and recreated. Covered in depth in Chapter 7.
- bullet**Ingress** — a Kubernetes-native object describing how external HTTP/HTTPS traffic should be routed into Services inside the cluster. OpenShift historically added its own richer equivalent, the **Route**, before Ingress existed in upstream Kubernetes; both exist today and are covered in Chapter 6.
- bullet**Label and Selector** — labels are key/value tags attached to objects (e.g., `app=frontend`); selectors are queries that other objects use to find things by their labels (e.g., a Service finds its Pods via a label selector). **Why:** this is the mechanism that lets Kubernetes stay loosely coupled — nothing hardcodes Pod names, everything queries by label.

3.5 The reconciliation loop — the single idea that explains everything

Every controller in Kubernetes follows the same pattern, called a **reconciliation loop**: watch the desired state stored in etcd (via the API server), compare it to the actual state observed in the cluster, and take action to close any gap — repeatedly, forever. A Deployment saying "3 replicas" is a desired state; if you manually delete one Pod, the ReplicaSet controller notices the gap (2 actual vs 3 desired) within moments and creates a replacement. You never issue a command to "restart pod X" — you declare what you want, and controllers continuously drive reality toward it. This declarative model, rather than a series of imperative commands, is why Kubernetes (and OpenShift) is considered self-healing.

Chapter 4 — What OpenShift Adds on Top of Kubernetes

If Chapter 3 was the engine, this chapter is everything Red Hat built around the engine to make it a usable, secure, supported product. Interviewers frequently ask "what's the difference between Kubernetes and OpenShift" — this chapter is your complete answer.

4.1 Projects (OpenShift's Namespace, with extras)

A **Project** is a Kubernetes Namespace with additional OpenShift metadata: an owner/requester annotation, default network isolation behaviour, and default resource quotas that an administrator can template across every new project automatically. Any `oc new-project myapp` command is really creating a Namespace plus this extra layer.

4.2 Security Context Constraints (SCC) — OpenShift's stricter default security

Upstream Kubernetes, by default, will happily let a container run as the root user (UID 0) inside the container, use host networking, or mount host paths, unless an administrator explicitly writes a Pod Security Policy/Standard to stop it. OpenShift took the opposite default position from day one: **no Pod may run as root, or do anything privileged, unless explicitly granted permission**, via an object called a **Security Context Constraint (SCC)**.

The default SCC (`restricted` or, in newer versions, `restricted-v2`) forces every container to run as an arbitrary, randomly-assigned non-root user ID unless the image/deployment is granted a different SCC.

This is the single most common "gotcha" for people bringing Docker images from elsewhere into OpenShift: an image that assumes it can write to `/var/log` as root, or bind to port 80 (ports below 1024 need root), will fail until it's fixed to run as non-root or granted a specific SCC.

Why this exists: in a shared, multi-tenant cluster, letting any team's container run as root is a serious security risk — a compromised container running as root has a much larger blast radius. OpenShift's philosophy is "secure by default, loosen deliberately," rather than Kubernetes' "open by default, tighten deliberately."

4.3 Routes — OpenShift's original way to expose applications

Before Kubernetes had a standard "Ingress" object, OpenShift already had **Routes**: an object that exposes a Service to the outside world under a real hostname, with OpenShift automatically handling the underlying HAProxy-based router, TLS termination, and even automatic hostname generation (`myapp-myproject.apps.cluster.example.com`). Kubernetes Ingress was added later industry-wide and is also supported in OpenShift today, but Route remains the native, more feature-rich option and is still what most `oc expose` commands create. Chapter 6 covers this in detail.

4.4 Built-in Integrated Image Registry

Every OpenShift cluster comes with its own internal container image registry, running as a workload inside the cluster itself, addressable as `image-registry.openshift-image-registry.svc:5000` from inside the cluster. When you build an application with OpenShift's build system (next section), the resulting image is pushed here automatically — you often never need an external registry like Docker Hub at all for your own applications.

4.5 Source-to-Image (S2I) and BuildConfig — build without writing a Dockerfile

Source-to-Image (S2I) is a Red Hat build strategy that takes your application source code straight from a Git repository and a pre-made "builder image" for your language (e.g., a Node.js builder, a Python builder) and produces a runnable container image — without you writing a Dockerfile at all. A **BuildConfig** is the OpenShift object that describes this build: where the source is, which builder/strategy to use, and where to push the result (usually the internal registry). **Why this exists:** it lowers the barrier for application developers who should not need to become container/Dockerfile experts just to deploy code; it also produces more consistent, centrally-maintained, patched base images across an organisation.

• `BuildConfig` → describes *how* to build.

• `Build` → one actual execution/run of that BuildConfig (like a CI job run).

• `ImageStream` → a special OpenShift object that tracks all the versions ("tags") of an image over time and can automatically trigger a new deployment when a new image lands — think of it as a smart pointer/alias system sitting in front of raw image tags.

4.6 DeploymentConfig vs Deployment

OpenShift originally had its own object, **DeploymentConfig (DC)**, predating Kubernetes' own Deployment object, with OpenShift-specific extras like automatically redeploying when an ImageStream updates and richer built-in lifecycle hooks. Kubernetes' native **Deployment** object later matured to cover almost all of the same ground and is now the officially recommended, forward-looking choice — DeploymentConfig is considered legacy in current OpenShift versions. **Why you need to know both names:** a large amount of existing real-world OpenShift documentation, older codebases, and interview questions still reference DeploymentConfig, so you must recognise it even though new work should use Deployment.

4.7 The Web Console

A full graphical dashboard (reachable at a URL like `https://console-openshift-console.apps.cluster.example.com`) for browsing projects, deploying applications from a catalog or Git repository, viewing logs and metrics, managing users and RBAC, and administering the cluster — all without touching the command line. Plain Kubernetes has no equivalent built in (the community `kubernetes-dashboard` add-on is not part of Kubernetes itself and is far less capable).

4.8 OAuth-based Authentication and Identity Providers

OpenShift ships a built-in **OAuth server** integrated with the cluster, supporting login via LDAP, GitHub, Google, HTTPasswd (a simple username/password file — what you will use in your homelab), and enterprise identity providers, all mapped automatically into Kubernetes-native user and group objects that RBAC (Role-Based Access Control) rules can reference. Plain Kubernetes leaves authentication entirely up to whoever installs the cluster.

4.9 Operators and the Operator Lifecycle Manager (OLM)

An **Operator** is a piece of software that packages not just an application, but the *operational knowledge* of running that application — installing it, upgrading it, backing it up, recovering it from failure — encoded as Kubernetes-native automation. The **Operator Lifecycle Manager (OLM)** is the OpenShift subsystem that lets administrators discover, install, and auto-update Operators from a curated catalog called **OperatorHub**, directly from the web console, the same way you would install an app from a phone's app store. Chapter 10 is dedicated to this because it is one of OpenShift's biggest differentiators and one of the most-asked-about topics in interviews.

4.10 Summary table — Kubernetes vs OpenShift

Capability	Plain/Upstream Kubernetes	OpenShift
Container runtime	Pluggable (containerd, CRI-O, etc.)	CRI-O, standardised
Default root access in containers	Allowed unless restricted	Blocked by default (SCC)
Exposing apps externally	Ingress only (needs a controller add-on)	Route (built-in) + Ingress
Image registry	Not included	Built-in, integrated
Building images from source	Not included (needs external CI)	Built-in (S2I, BuildConfig)
Web console	Not included by default	Full-featured, built-in
Authentication	Bring-your-own	Built-in OAuth server
Software marketplace	Helm charts (manual)	OperatorHub + OLM (managed)
Support model	Community / self-support or vendor-specific	Red Hat enterprise support (paid) or OKD (free community)
Underlying object model	Kubernetes API	100% Kubernetes API + additional OpenShift APIs

Chapter 5 — OpenShift Architecture, End to End

This chapter walks through the full architecture as one continuous story: what happens physically and logically from the moment you power on the cluster to the moment a user's web browser receives a response from your application.

5.1 The three node roles

Control plane (master) nodes — run the API server, etcd, scheduler, controller manager, and the OpenShift-specific control-plane operators. Recommended minimum in production: 3, for quorum-based high availability. In a learning/SNO setup: 1 node plays this role too.

Worker (compute) nodes — run your actual application Pods. Scaled up or down based on load.

Infrastructure nodes (optional, common in production) — a dedicated pool of nodes that run cluster-support workloads (the router/ingress, the internal registry, monitoring, logging) so that these "housekeeping" workloads don't compete for resources with application Pods, and so licensing/subscription counting can sometimes be optimised. Not required in a small homelab.

5.2 RHCOS — the operating system every node runs

RHCOS (Red Hat Enterprise Linux CoreOS) is a minimal, container-optimised, immutable operating system that every node in a standard OpenShift installation runs. "Immutable" here means the OS filesystem is not meant to be manually edited node-by-node — instead, configuration changes are delivered as a whole new OS image via the **Machine Config Operator**, and every node of the same "pool" (control-plane or worker) stays byte-for-byte identical. This is a deliberate design choice: it removes configuration drift (the classic problem of "node 7 has a slightly different patch level than node 3 because someone manually fixed something on it once") entirely. RHCOS uses **Ignition** (a first-boot provisioning system) to apply its initial configuration and **rpm-ostree** to apply OS updates as atomic, rollback-able transactions, rather than traditional package-by-package `yum`/`dnf` updates.

Because you are building on ESXi with limited resources, you should know upfront: full self-managed OpenShift with RHCOS on every node is comparatively heavy (each control-plane node alone is recommended to have 4 vCPU / 16 GB RAM by Red Hat's official minimums). Chapter 13 and 14 explain the lighter-weight, homelab-realistic alternatives (CRC, OKD SNO) that still teach you the same underlying concepts.

5.3 Operators all the way down — the Cluster Version Operator

A distinguishing architectural trait of OpenShift (compared to a manually-assembled Kubernetes cluster) is that almost every cluster capability — networking, DNS, authentication, the image registry, monitoring, ingress — is itself managed by a dedicated **Cluster Operator**, and all of those Cluster Operators are in turn managed by one top-level **Cluster Version Operator (CVO)**. The CVO is what makes an OpenShift cluster upgrade a single, coordinated, largely automatic action ("upgrade the cluster from 4.15 to 4.16") instead of dozens of separate manual component upgrades. Running `oc`

get clusteroperators` on a live cluster shows you this entire list and their health — it is one of the first commands worth memorising.

5.4 The request path — from a browser to your Pod, step by step

- 1 A user's browser resolves `myapp-myproject.apps.cluster.example.com` via DNS to the IP address of the cluster's **router** (HAProxy-based, running as Pods, fronted by a load balancer or, in a small setup, directly by a node's IP).
- 2 The **Route** object tells the router which internal Service to forward requests for that hostname to, and whether/how TLS should be handled (edge termination, passthrough, or re-encrypt — explained in Chapter 6).
- 3 The router forwards the request to the Service's virtual IP.
- 4 **kube-proxy** rules on the node route that virtual IP traffic to one of the healthy Pod IP addresses backing the Service (chosen via the Service's label selector).
- 5 The request arrives at your container's process inside the Pod, which handles it and returns a response, which flows back out the same path.

5.5 How a Pod actually gets scheduled and started

- 1 You (or a Deployment's ReplicaSet controller) create a Pod object via `oc apply` or the console — this is really an HTTP request to the **API server**.
- 2 The API server authenticates you, checks **RBAC** to confirm you're allowed to create Pods in this Project, validates the object against the schema, and writes it to **etcd** with a status of "Pending, unscheduled."
- 3 The **scheduler** notices the new unscheduled Pod, evaluates every node's available CPU/memory, taints/tolerations, and affinity rules, and picks the best node, writing that decision back through the API server.
- 4 The **kubelet** on the chosen node notices (via watching the API server) that a Pod has been assigned to its node, and instructs **CRI-O** to pull the required image (from the internal registry or elsewhere) if not already cached, then start the container(s).
- 5 The kubelet continuously reports Pod status (Running, Ready, CrashLoopBackOff, etc.) back to the API server, which is what you see when you run `oc get pods`.

5.6 Multi-cluster / hybrid concepts you will hear about

Advanced Cluster Management (ACM) — Red Hat's product for managing many OpenShift clusters (on-prem, multiple clouds, edge) from one place: policies, application distribution, and observability across a fleet.

Hybrid cloud — the common real-world pattern of running OpenShift both on-premises (like your ESXi homelab) and in a public cloud (AWS, Azure, GCP, IBM Cloud), using the same tooling and APIs in both places so teams don't need two completely different skill sets.

Chapter 6 — Acronym Glossary (Read Once, Reference Forever)

This is intentionally placed early — every acronym used anywhere else in this book is defined here, in one place, so you never have to hunt for a definition mid-chapter.

Acronym	Full form	Plain-English meaning
K8s	Kubernetes (8 letters between "K" and "s")	The container orchestration engine OpenShift is built on
OCP	OpenShift Container Platform	Red Hat's commercial, supported OpenShift product
OKD	Origin Community Distribution	The free, community edition of OpenShift (upstream of OCP)
CRC	CodeReady Containers	A tool that runs a full mini single-node OpenShift on your own laptop/PC
SNO	Single Node OpenShift	A production-grade OpenShift cluster running on exactly one node
RHCOS	Red Hat Enterprise Linux CoreOS	The minimal, immutable OS every standard OpenShift node runs
RHEL	Red Hat Enterprise Linux	Red Hat's general-purpose enterprise Linux distribution
CRI	Container Runtime Interface	The Kubernetes standard API for plugging in a container runtime
CRI-O	(name derived from CRI + OCI)	The lightweight container runtime OpenShift uses under the hood
OCI	Open Container Initiative	The industry-standard spec for container images and runtimes
CVO	Cluster Version Operator	The top-level Operator that manages all other Operators/upgrades
OLM	Operator Lifecycle Manager	The subsystem that installs/updates Operators from a catalog

Acronym	Full form	Plain-English meaning
CRD	Custom Resource Definition	A way to add new, custom object types to the Kubernetes API
SCC	Security Context Constraint	OpenShift's mechanism controlling what a Pod is allowed to do
RBAC	Role-Based Access Control	The permission system deciding who can do what in the cluster
S2I	Source-to-Image	Building a container image directly from source code, no Dockerfile
DC	DeploymentConfig	OpenShift's original (now legacy) app rollout object
PV	PersistentVolume	A piece of real storage made available to the cluster
PVC	PersistentVolumeClaim	A request by an app for a PersistentVolume
CSI	Container Storage Interface	The standard plugin API connecting Kubernetes to storage systems
SDN	Software-Defined Networking	Networking that is configured entirely in software, not physical switches
OVN	Open Virtual Network	The modern networking technology OpenShift uses by default
CNI	Container Network Interface	The standard plugin API for Kubernetes cluster networking
DNS	Domain Name System	Translates hostnames into IP addresses
TLS	Transport Layer Security	The encryption protocol behind HTTPS
HA	High Availability	Design that keeps a system running despite individual failures
IPI	Installer-Provisioned Infrastructure	OpenShift installer creates the VMs/servers itself

Acronym	Full form	Plain-English meaning
UPI	User-Provisioned Infrastructure	You create the VMs/servers yourself, installer only configures OpenShift
API	Application Programming Interface	The structured way software components talk to each other
CLI	Command-Line Interface	Interacting with software by typing commands (e.g., <code>`oc`</code> , <code>`kubectl`</code>)
YAML	YAML Ain't Markup Language	The human-readable text format almost all Kubernetes/OpenShift objects are written in
JSON	JavaScript Object Notation	Another structured data format, used internally alongside YAML
CI/CD	Continuous Integration / Continuous Delivery (or Deployment)	Automating build, test, and release of software
GitOps	(Git + Operations)	Managing infrastructure/app state by syncing it from a Git repository
ArgoCD	Argo Continuous Delivery	The most common GitOps tool used with OpenShift
Tekton	(Māori word for "carpenter/craftsman," project name)	Kubernetes-native CI/CD pipeline engine used by OpenShift Pipelines
EFK	Elasticsearch, Fluentd, Kibana	A classic open-source log storage/search/visualisation stack
PLG / Loki stack	Promtail, Loki, Grafana	The newer, lighter-weight logging stack OpenShift is moving toward
VM	Virtual Machine	A simulated computer running its own OS on a hypervisor
ESXi	Elastic Sky X Integrated	VMware's bare-metal hypervisor (what your homelab host runs)
vSphere	(VMware's product family name)	VMware's suite for managing ESXi hosts and VMs, incl. vCenter

Acronym	Full form	Plain-English meaning
vCenter	(VMware's central management server)	The management server that controls one or more ESXi hosts
OVA/OVF	Open Virtual Appliance / Open Virtualization Format	A packaged, importable VM template format
RAM	Random Access Memory	The computer's working memory
SSD	Solid State Drive	Fast, non-mechanical storage
CPU / vCPU	Central Processing Unit / virtual CPU	The processor (or its virtualized slice in a VM)
NIC	Network Interface Card	The hardware/virtual device providing network connectivity
LB	Load Balancer	Distributes incoming traffic across multiple backend targets
HAProxy	High Availability Proxy	The specific software OpenShift's built-in router is based on
OAuth	Open Authorization	The standard protocol OpenShift's login/authentication system uses
LDAP	Lightweight Directory Access Protocol	A standard protocol for centralized user directories (e.g., corporate logins)
SSH	Secure Shell	Encrypted remote command-line access to another machine
SELinux	Security-Enhanced Linux	A Linux kernel security module enforcing strict access rules
EtcD	(from "/etc" + "distributed")	The key-value database storing all cluster state
YAML manifest	—	A YAML file describing one or more Kubernetes/OpenShift objects
MCO	Machine Config Operator	Manages OS-level configuration across all RHCOS nodes
MTU	Maximum Transmission Unit	The largest packet size a network link allows

Acronym	Full form	Plain-English meaning
NAT	Network Address Translation	Rewriting IP addresses as traffic crosses a network boundary
VIP	Virtual IP address	A shared IP address that can move between machines/services
SRE	Site Reliability Engineering	A discipline/role focused on running reliable production systems
POC	Proof of Concept	A small trial project to test whether an idea/technology works
SLA	Service Level Agreement	A formal commitment about uptime/performance
SIG	Special Interest Group	A community sub-group within an open-source project (e.g., Kubernetes SIGs)

Tip: don't try to memorise this table today. Read it once now, then come back to it any time a later chapter uses an acronym you've forgotten — that's the entire point of putting it early and giving it its own chapter.

Chapter 7 — Networking in OpenShift

7.1 Why cluster networking is hard in the first place

In a normal network, every machine gets one IP address, assigned once, and that's that. In a Kubernetes/OpenShift cluster, Pods are created and destroyed constantly, and each one needs its own IP address, reachable from every other Pod on every other node, without you manually configuring routes every time. This is solved by a plugin architecture called the **Container Network Interface (CNI)**.

7.2 OVN-Kubernetes — the default CNI plugin

Modern OpenShift defaults to **OVN-Kubernetes**, built on **Open Virtual Network (OVN)**, itself built on **Open vSwitch (OVS)**. Every node runs a virtual switch, and these switches are coordinated by a small distributed database so the whole cluster behaves like one flat virtual network. Earlier OpenShift versions used **OpenShift SDN**, now deprecated in favour of OVN-Kubernetes.

7.3 The four networking problems Kubernetes/OpenShift networking must solve

- bullet **Pod-to-Pod on the same node** — a local virtual bridge.
- bullet **Pod-to-Pod across nodes** — the CNI plugin's overlay network.
- bullet **Pod-to-Service** — kube-proxy (or OVN-Kubernetes's equivalent) redirects traffic sent to a stable Service IP toward healthy backing Pods.
- bullet **External-to-Service** — Routes/Ingress plus the router.

7.4 Services — the four types

- bullet **ClusterIP** (default) — internal-only virtual IP.
- bullet **NodePort** — additionally exposes a fixed port (30000-32767) on every node's real IP.
- bullet **LoadBalancer** — asks the infrastructure provider to provision an external load balancer.
- bullet **Headless Service** — no virtual IP; DNS returns individual Pod IPs directly, used by stateful apps needing to address each replica.

7.5 Routes in depth

- bullet **Edge termination** — TLS decrypted at the router; router-to-Pod traffic is plain HTTP.
- bullet **Passthrough termination** — TLS passes straight through; the Pod handles its own certificates.
- bullet **Re-encrypt termination** — router decrypts, then re-encrypts with a different certificate before forwarding.

7.6 Ingress vs Route

OpenShift automatically creates a Route behind the scenes for every Ingress object you create. Use Ingress for portability across Kubernetes distributions; use Route directly for the full native feature set.

7.7 NetworkPolicy

By default every Pod can talk to every other Pod in every namespace. A **NetworkPolicy** restricts this — e.g., "only Pods labelled `role=frontend` may reach Pods labelled `role=backend`" — enforcing isolation at the network layer, on top of RBAC.

Chapter 8 — Security: RBAC, SCC, Projects, and Secrets

8.1 Why security is a first-class chapter

A cluster is shared infrastructure — multiple teams and applications on the same hardware. OpenShift's security model operates at several independent layers, and knowing which layer stops which problem is the mark of real understanding.

8.2 Authentication vs Authorization

bullet**Authentication** — "who are you?" — OpenShift's built-in OAuth server plus an identity provider (HTPasswd, LDAP, GitHub, corporate SSO).

bullet**Authorization** — "what are you allowed to do?" — **RBAC**.

8.3 RBAC building blocks

bullet**Role** — namespaced permissions.

bullet**ClusterRole** — cluster-wide permissions.

bullet**RoleBinding** — attaches a Role to a user/group in one namespace.

bullet**ClusterRoleBinding** — attaches a ClusterRole cluster-wide.

bullet**ServiceAccount** — an identity for a process/Pod rather than a human.

This split lets an admin say "the CI system's ServiceAccount can deploy to dev only," without touching human accounts or leaking access elsewhere.

8.4 SCC in more depth

bullet`restricted-v2` (default) — no root, no host access, random UID.

bullet`nonroot-v2` — non-root required, but app-specified UID allowed.

bullet`anyuid` — any UID including root, still no host access.

bullet`privileged` — essentially unrestricted; reserved for infra components.

8.5 Secrets and their real-world limitations

A **Secret** is base64-encoded, not encrypted — anyone with API read access can decode it. Real protection comes from RBAC, etcd encryption at rest, or an external secret manager like HashiCorp Vault.

8.6 Image security

Image Policies can require images to come from approved, signed sources, and integrate with vulnerability scanners to block images with known critical CVEs (Common Vulnerabilities and Exposures).

Chapter 9 — The Application Lifecycle: From Code to Running Pod

9.1 Three ways to get an application running

- 1 Deploy an existing container image directly via a Deployment.
- 2 Build from source with S2I — OpenShift compiles your Git repo into an image automatically.
- 3 Deploy from a Template or Helm chart — a pre-packaged, parameterised application stack.

9.2 Templates vs Helm

OpenShift's original packaging was the **Template** object; the wider ecosystem standardised on **Helm**, which OpenShift fully supports and which is now the more portable choice.

9.3 A full walk-through

```
oc new-project demo
oc new-app https://github.com/example/simple-node-app --name=myapp
oc logs -f bc/myapp
oc expose svc/myapp
oc get route myapp
```

``new-project`` creates a Project. ``new-app`` detects the language, picks an S2I builder, and creates a BuildConfig, ImageStream, Deployment, and Service in one command. ``logs -f bc/myapp`` follows the build live. ``expose svc/myapp`` creates a Route. ``get route myapp`` shows the URL. New commits trigger a rebuild and a zero-downtime rolling update.

9.4 Rolling updates, readiness, and liveness

Readiness probe — "ready for traffic right now"; a failing Pod is pulled from the Service's target pool.

Liveness probe — "process is alive and healthy"; a failing Pod is killed and restarted.

Forgetting proper readiness probes is one of the most common real production bugs — traffic keeps flowing to broken Pods.

9.5 Rollbacks

``oc rollout undo deployment/myapp`` reverts to the previous working version in seconds — a critical safety net and a very common real-incident scenario.

Chapter 10 — Operators and the Operator Lifecycle Manager (OLM)

10.1 The problem Operators solve

Installing complex software (a database, a message queue, an AI serving stack) is not just "start a container" — it's setup, backup, upgrades, failure recovery, and tuning. An **Operator** encodes that operational knowledge as software: a controller watching a **Custom Resource (CR)** and continuously driving the cluster toward the desired state.

10.2 Custom Resource Definitions (CRDs)

A **CRD** adds a brand-new object type to the Kubernetes API — e.g., `PostgresCluster``. Once installed, `oc get postgresclusters`` works like any built-in object. The Operator is the controller watching and acting on those objects.

10.3 OperatorHub and OLM in practice

From the console's **OperatorHub**, you browse certified/community Operators, install into a namespace and update channel, and OLM installs CRDs, grants exactly the RBAC the Operator declares needing, and keeps it auto-updated.

10.4 Why this matters for your AI/ML background

Red Hat OpenShift AI is delivered as Operators (plus dependencies like the NVIDIA GPU Operator). Understanding Operators is the direct mechanism by which a notebook/training/serving platform gets installed and kept healthy on your cluster.

Chapter 11 — CI/CD on OpenShift

11.1 Why CI/CD and OpenShift go together

CI/CD automates build, test, and release of software. OpenShift is frequently the deployment target of a pipeline, and increasingly also runs the pipeline engine itself.

11.2 OpenShift Pipelines (built on Tekton)

Tekton pipelines are themselves Kubernetes objects (`Pipeline``, `Task``, `PipelineRun``, `TaskRun``); each step runs as its own Pod. **OpenShift Pipelines** is Red Hat's Operator-installed packaging of Tekton, giving CI/CD the same RBAC, storage, and scaling model as applications.

11.3 GitOps with ArgoCD (OpenShift GitOps)

GitOps treats Git as the single source of truth; **ArgoCD** (packaged as **OpenShift GitOps**) continuously reconciles the live cluster to match Git. Manual undocumented changes get reverted on the next sync — a deliberate feature forcing every change to be an auditable Git commit.

11.4 Jenkins

Jenkins was the classic CI engine paired with OpenShift and is still common in existing enterprise pipelines, even as new projects favour Tekton.

Chapter 12 — Monitoring and Logging

12.1 Monitoring stack

bullet**Prometheus** — scrapes metrics from every cluster component and, optionally, your apps.

bullet**Alertmanager** — evaluates alerting rules and routes alerts (email, Slack, PagerDuty).

bullet**Grafana / console dashboards** — visualises metrics.

Built in by default so an admin has day-one visibility without standing up monitoring separately first.

12.2 Logging: from EFK toward Loki

OpenShift's Cluster Logging Operator historically shipped **EFK** (Elasticsearch, Fluentd, Kibana); newer versions default to lighter **Loki** plus **Vector**/**Fluentd**, visualised in the console. Loki indexes only metadata (labels), not full text, making it cheaper to run at scale than Elasticsearch.

12.3 The three pillars of observability

bullet**Metrics** — numeric time-series — Prometheus.

bullet**Logs** — discrete text events — the logging stack.

bullet**Traces** — a request's path across services with timing — distributed tracing (Jaeger/OpenTelemetry), most useful in microservices.

Chapter 13 — Storage: PV, PVC, StorageClass, and CSI

13.1 Why Pods need special help with storage

A container's own filesystem is ephemeral — the moment the container is deleted or rescheduled to another node, anything it wrote is gone. For stateless applications (most web front ends) this is fine. For anything that must remember data — a database, a file upload service, a Jupyter notebook's saved work — you need storage that survives Pod restarts and can, if needed, be reattached to a replacement Pod even on a different node.

13.2 The core objects

- bullet**PersistentVolume (PV)** — represents an actual piece of storage capacity in the cluster (a disk, an NFS share, a vSAN volume, etc.), created either by an administrator manually or automatically by a provisioner.
- bullet**PersistentVolumeClaim (PVC)** — a request made by an application: "I need 10 GB of storage that behaves like this." Kubernetes matches (binds) the PVC to a suitable PV.
- bullet**StorageClass** — a template describing *how* to dynamically create new PVs on demand (which underlying storage backend, which performance tier, which access mode), so that most of the time an application developer just creates a PVC and the matching PV is created automatically — nobody has to pre-provision storage by hand.
- bullet**Container Storage Interface (CSI)** — the standard plugin API letting Kubernetes/OpenShift talk to virtually any storage backend (vSphere datastores, AWS EBS, Ceph, NFS, local disks) through a common interface, so the platform doesn't need backend-specific code baked in.

13.3 Access modes

- bullet**ReadWriteOnce (RWO)** — mountable read/write by a single node at a time (most block storage, including typical vSphere-backed disks).
- bullet**ReadWriteMany (RWX)** — mountable read/write by many nodes simultaneously (needs a shared filesystem backend like NFS or Ceph).
- bullet**ReadOnlyMany (ROX)** — mountable read-only by many nodes.

Why this matters for your homelab: a single ESXi datastore-backed PV is naturally RWO. If you ever need two Pods on two different nodes to share the same writable files, you'll need an NFS server or similar shared storage layered on top, not raw vSphere disks directly.

13.4 vSphere-specific storage in OpenShift

When OpenShift is installed on vSphere/ESXi, the **vSphere CSI driver** (installed automatically by the vSphere-integrated install path, or configurable manually) lets PVCs dynamically provision new virtual disks directly on your ESXi datastore, without you ever touching the vSphere client to create disks by hand.

13.5 Local storage for small/home setups

For a small single-node or homelab cluster where you don't need a full CSI-backed dynamic provisioner, the **Local Storage Operator** or a simple `hostPath`-backed manual PV can be used to point directly at a folder on the node's own disk — simpler to set up, but it ties that data permanently to that one physical node, which is an acceptable trade-off for a learning environment.

Chapter 14 — Installation Methods: The Big Picture Before You Install Anything

14.1 The two fundamental installation philosophies

- bullet**Installer-Provisioned Infrastructure (IPI)** — you point the OpenShift installer at a supported infrastructure provider's API (AWS, Azure, GCP, or vSphere/vCenter), and the installer itself creates every VM, configures networking, and bootstraps the whole cluster automatically. Fastest and least error-prone, but requires the installer to have API access to create resources itself (e.g., real vCenter credentials with permission to create VMs).
- bullet**User-Provisioned Infrastructure (UPI)** — you create every VM yourself (in vSphere, that means manually cloning an OVA template several times), set up your own DNS records and load balancer, and the installer only configures OpenShift software on top of machines you already built. More manual work, but works in constrained or non-standard environments, and teaches you far more about what's actually required underneath.

For a home ESXi setup without a fully API-integrated vCenter (many home labs run standalone ESXi without vCenter Server at all), **UPI is usually the realistic path** for a full multi-node cluster, while IPI's vSphere path specifically requires vCenter. This is directly relevant to your planning in the next chapter.

14.2 Full multi-node OpenShift vs the lighter-weight alternatives

- bullet**Full OpenShift Container Platform (OCP), multi-node** — the real production architecture: 3 control-plane + N worker nodes, each RHCOS, each needing real CPU/RAM/disk. Red Hat's official minimums are demanding (control-plane nodes: 4 vCPU / 16 GB RAM / 100 GB disk each, as a baseline) — well beyond a single 32 GB host if you tried to run 3 control-plane VMs plus workers simultaneously.
- bullet**Single Node OpenShift (SNO)** — one single RHCOS node plays control-plane and worker simultaneously. Still a "real" OpenShift cluster (same APIs, same Operators, same behaviour), just with no high availability. Minimum footprint is far friendlier: officially around 8 vCPU / 16 GB RAM / 120 GB disk minimum, though it can be made to run smaller for learning purposes with reduced expectations.
- bullet**CodeReady Containers (CRC)** — a purpose-built tool that runs a complete, real single-node OpenShift cluster inside one VM on your own laptop/desktop, specifically designed for local development and learning. Officially recommended minimum: 4 vCPU / 9 GB RAM / 35 GB free disk for the CRC VM itself. This is comfortably achievable on your 32 GB ESXi host or even directly on your laptop.
- bullet**OKD (Origin Community Distribution)** — the free, community-maintained upstream of OpenShift, functionally very close to OCP but without Red Hat's paid support and without the RHEL subscription requirement, making it an excellent zero-cost way to learn the real architecture, including full multi-node UPI installs.

14.3 Why the free/community route makes sense for you specifically

OCP requires a Red Hat subscription (free Red Hat Developer subscriptions exist and are sufficient for learning and even small non-production use, so cost is not actually a hard blocker) but OKD requires no subscription of any kind and installs almost identically, using the same `oc`/`openshift-install`` tooling and the same underlying architecture described in every earlier chapter of this book. Everything you learn on OKD transfers directly to OCP in a job. Chapter 15 gives you a concrete, resource-aware recommendation between CRC, SNO, and a small multi-node OKD cluster for your exact hardware.

Chapter 15 — Planning Your Cluster on Your Actual ESXi Box

15.1 Doing the resource math honestly

Your host: 32 GB RAM, 320 GB SSD, running ESXi. Let's be honest about what fits, because pretending a full 3-master + 3-worker OCP cluster fits in 32 GB would set you up to fail on install day.

Setup	vCPU need	RAM need	Disk need	Realistic on your box?
Full OCP, 3 control + 2 worker	~20+ vCPU	80-100+ GB	500+ GB	No — needs far more RAM than you have
OKD, 3 control + 1 worker (UPI, trimmed)	~12-16 vCPU	~48-56 GB even trimmed	300+ GB	Tight to impossible without swapping/oversubscribing badly
Single Node OpenShift (SNO) / OKD SNO	6-8 vCPU (can oversubscribe)	16-20 GB	120-150 GB	**Yes — comfortably, with room for your other VMs**
CodeReady Containers (CRC)	4 vCPU	9-10 GB	35-40 GB	**Yes — very comfortably, fastest to get running today**

ESXi lets you oversubscribe vCPU (assign more vCPUs across VMs than your host has physical cores) far more safely than you can oversubscribe RAM — memory pressure causes hypervisor swapping and severe slowdowns, while modest CPU oversubscription usually just adds scheduling latency. Treat 32 GB of RAM as close to a hard ceiling once you subtract a few GB for ESXi itself and any other VMs you keep running (like your RHEL 9 Ansible control node, which you already have).

15.2 The recommended path, in order

- 1 ****Start with CRC on your laptop or as a VM on ESXi.**** It gets you a real OpenShift cluster running today, with almost no infrastructure planning, and every ``oc`` command, every Route, every Operator concept from this book works identically inside it. Use this while you're still internalising Chapters 1-12.
- 2 ****Once comfortable, build a Single Node OKD cluster as a dedicated VM on your ESXi host.**** This is a "real" cluster install (bootstrap process, Ignition, RHCOS) rather than a wrapped dev tool, and is the natural next step in difficulty — genuinely representative of a production SNO deployment used in real edge/telco use cases (Chapter 19).

- 3 ****Only if you later get access to more hardware (a second physical host, or a cloud trial), attempt a full 3+2 multi-node cluster.**** There is no shortcut around the RAM requirement for true multi-master HA — it is not a configuration you can meaningfully shrink, because etcd and the API server genuinely need that headroom to behave correctly under real conditions.

15.3 Network planning on ESXi

- bullet** Decide whether your OpenShift VMs sit on your existing home LAN subnet (simpler DNS/routing, but shares address space with your other devices) or on a dedicated virtual switch/port group in ESXi (cleaner isolation, but you must handle routing yourself, which is exactly what your Tailscale relay-VM setup is already positioned to help with).
- bullet** OpenShift needs a small number of DNS records to exist and resolve correctly no matter which install path you choose: an API endpoint (e.g., ``api.ocp4.lab.local``), a wildcard for application routes (e.g., ``*.apps.ocp4.lab.local``), and for multi-node UPI, an internal API record too. For a single VM/SNO setup, the simplest reliable option is running a small ****BIND DNS**** instance (which, based on your recent homelab work, you already have hands-on experience setting up) or using ``dnsmasq``, rather than trying to edit every client's hosts file.
- bullet** If you plan to reach the cluster's web console/API from outside your home network (e.g., while at the Prodevans office), your existing Tailscale subnet-routing setup is the right tool — route the cluster's subnet through Tailscale rather than exposing the ESXi host directly to the internet.

15.4 Disk planning across 320 GB

A sensible allocation for a SNO/OKD single-node build, leaving headroom for your other VMs (RHEL 9 Ansible control node, DNS VM, etc.):

Item	Approx. size
ESXi itself + scratch	~10-15 GB (usually on a separate boot device already)
SNO/OKD node VM disk	120-150 GB
CRC VM disk (if kept alongside for quick experiments)	40 GB
Existing RHEL 9 Ansible control node	(whatever you've already allocated)
Existing DNS / BIND VM	(whatever you've already allocated)
Free buffer for snapshots, ISO storage, extra PVs	remaining

Chapter 16 — Step-by-Step: Installing on Your ESXi Homelab

16.1 Path A — CRC (do this first, today)

1. On your laptop (or a Linux VM on ESXi with 4 vCPU / 10 GB RAM / 40 GB disk):
 - Create a free Red Hat Developer account at developers.redhat.com
 - Download the CRC binary for your OS and the "pull secret" from console.redhat.com/openshift/create/local
2. Install and initialise:


```
crc setup
```
3. Configure resources before first start (adjust to your headroom):


```
crc config set cpus 4
crc config set memory 10240
crc config set disk-size 40
```
4. Start the cluster (first run downloads the CRC image, several GB):


```
crc start --pull-secret-file ~/pull-secret.txt
```
5. Log in as admin once it finishes:


```
eval $(crc oc-env)
oc login -u kubeadmin -p <password-shown-at-end-of-crc-start>
```
6. Open the console URL also printed at the end of `crc start` in your browser.

At this point every command and object from Chapters 3, 4, 6, 7, 9, and 10 works exactly as described — this is a genuine OpenShift cluster, just scoped to one VM.

16.2 Path B — Single Node OKD on ESXi (the "real install" milestone)

This is a UPI-style (Chapter 14) install since a bare ESXi host without vCenter cannot run IPI's vSphere path. The shape of the process:

- 1 ****Prepare DNS first, before touching the installer.**** Using your existing BIND setup (or a fresh small DNS VM), create these records pointing at the IP you will assign to your single node:
 - bullet`api.ocp4.lab.local` → node IP
 - bullet`api-int.ocp4.lab.local` → node IP
 - bullet`\*.apps.ocp4.lab.local` → node IP
- 1 ****Download the OKD installer and client**** (`openshift-install` and `oc`) matching the OKD release you choose, on your RHEL 9 Ansible control node or your laptop — this machine only needs to *run the installer*, it does not become part of the cluster.
- 2 ****Generate an install-config.yaml****, answering prompts for base domain (`lab.local`), cluster name (`ocp4`), and — critically for SNO — setting `compute.replicas: 0` and `controlPlane.replicas: 1`, which tells the installer to build a single-node topology rather than the default 3+2.
- 3 ****Generate Ignition configs**** from that install-config:

```
openshift-install create single-node-ignition-config --dir=ocp4-install
```


- 1 ****Create one new VM in the ESXi/vSphere client**** with the specs from Chapter 15 (8 vCPU if you can spare it, 16-20 GB RAM, 120-150 GB thin-provisioned disk), attach the generated ``bootstrap-in-place-for-live-iso.ign`` via the installer-provided live ISO (mount it as the VM's CD/DVD), and boot the VM from it.
- 2 ****Watch installation progress**** from your installer machine:

```
openshift-install wait-for install-complete --dir=ocp4-install
```

The single node reboots itself part-way through (this is expected — it's applying RHCOS and its own Ignition-driven configuration) and finishes by itself.

- 1 ****Retrieve your admin credentials and log in:****

```
export KUBECONFIG=ocp4-install/auth/kubeconfig
oc whoami
oc get nodes
oc get clusteroperators
```

A healthy install shows one ``Ready`` node and every ClusterOperator eventually reaching ``Available: True``.

16.3 Common install-time pitfalls, specific to your setup

- bullet****DNS not resolving before the install starts**** is the single most common cause of a stuck OKD/OCP install — the bootstrap and node processes constantly re-resolve ``api-int`` and will hang indefinitely if it's wrong. Verify all three records resolve correctly from a separate machine before you boot the installer VM.
- bullet****Thin-provisioned disks filling the datastore silently**** — a 150 GB thin-provisioned disk doesn't reserve 150 GB up front, but if several of your homelab VMs are all thin-provisioned and all grow toward their limits at once, your 320 GB SSD can run out unexpectedly. Keep an eye on actual datastore free space in the ESXi/vSphere client, not just what each VM reports internally.
- bullet****Not enough contiguous free RAM at boot**** — if other VMs are already powered on and holding memory, ESXi may struggle to start a memory-hungry OpenShift VM even if the **total** looks sufficient on paper. Consider powering off less-essential lab VMs during the initial install.

Chapter 17 — Using VS Code as Your OpenShift Workstation

17.1 Why VS Code fits this workflow well

You already use VS Code with Remote-SSH into your RHEL 9 Ansible control node. The exact same pattern extends naturally to OpenShift work: you don't need OpenShift tooling installed on your Windows/Mac laptop at all — install it once on the Linux control node you already SSH into, and edit everything remotely.

17.2 The extensions worth installing

- bullet**Remote - SSH** (you already use this) — lets VS Code's editor UI run locally while all terminal commands, file access, and extensions execute on your remote RHEL 9 node.
- bullet**YAML** (by Red Hat) — schema-aware autocomplete and validation for Kubernetes/OpenShift YAML manifests; catches indentation and field-name mistakes before you ever run ``oc apply``.
- bullet**OpenShift Toolkit / Kubernetes** (by Red Hat / Microsoft) — adds a sidebar for browsing your cluster's Projects, Pods, and logs directly inside VS Code, and can generate boilerplate manifests.

17.3 A practical day-to-day loop

- 1 Open a VS Code Remote-SSH session into your RHEL 9 control node.
- 2 Open an integrated terminal; confirm ``oc whoami`` shows you're logged into the right cluster (CRC or your SNO OKD node).
- 3 Write or edit a YAML manifest (Deployment, Service, Route) with full schema validation from the YAML extension.
- 4 Apply it directly from the same terminal: ``oc apply -f myapp.yaml``.
- 5 Use the Kubernetes/OpenShift sidebar, or ``oc logs`/`oc get events``, to check the result without leaving the editor.
- 6 Commit the manifest to a Git repository — this is also your first natural step toward the GitOps workflow from Chapter 11.

17.4 Keeping kubeconfig organised

As you accumulate more than one cluster (CRC, SNO OKD, maybe a cloud trial later), keep each cluster's kubeconfig file separate and use ``export KUBECONFIG=~/.kube/config-sno`` (or merge them with ``oc config`` contexts) rather than overwriting one shared file — a common source of "why am I suddenly talking to the wrong cluster" confusion.

Chapter 18 — Day-2 Operations: Running the Cluster, Not Just Installing It

18.1 Why "Day 2" is its own category

"Day 1" is installation. "Day 2" is everything that happens for the rest of the cluster's life: upgrades, scaling, certificate rotation, backup, and recovering from failures. Most real operational skill — and most interview questions beyond the basics — live here.

18.2 Upgrades

Cluster upgrades are driven by the **Cluster Version Operator** (Chapter 5): `oc adm upgrade --to-latest`` (or a specific version) triggers a coordinated, largely automatic upgrade of every Cluster Operator in the correct dependency order. On a single-node cluster, expect the node to reboot during the process; on multi-node clusters, nodes are upgraded one at a time (cordoned, drained, updated, rebooted, uncordoned) so the cluster keeps serving traffic throughout.

18.3 Scaling

- bullet**Scaling an application** — `oc scale deployment/myapp --replicas=5``, or configure a **HorizontalPodAutoscaler (HPA)** to do it automatically based on CPU/memory/custom metrics.
- bullet**Scaling the cluster itself** — adding worker nodes (via a **MachineSet**, on IPI-managed infrastructure, or manually joining a new UPI node) to add capacity as workloads grow.

18.4 Backup and disaster recovery

Because **etcd** (Chapter 5) holds the entire cluster's state, etcd backup is the single most important Day-2 safety task: `oc adm`` scripts capture a consistent etcd snapshot alongside the static pod manifests needed to recover the control plane. For applications, application-level data (databases running as workloads) needs its own backup strategy (e.g., Operator-driven backups, Velero) since etcd backup only protects Kubernetes/OpenShift **object** state, not your application's actual business data sitting in a PVC.

18.5 Node lifecycle: cordon, drain, and maintenance

- bullet`oc adm cordon node1`` — marks a node as unschedulable for new Pods, without touching what's already running there.
- bullet`oc adm drain node1`` — safely evicts existing Pods off the node (respecting PodDisruptionBudgets so you don't take down all replicas of something at once), so it can be rebooted or maintained.
- bullet`oc adm uncordon node1`` — makes the node schedulable again once maintenance is done.

18.6 Certificate rotation

Most internal cluster certificates rotate automatically. Understanding that OpenShift manages this itself (rather than requiring manual `openssl` work) is more important at your current stage than the low-level mechanics — just know it exists as a Day-2 concern administrators watch for in cluster health dashboards.

Chapter 19 — Real-World Use Cases and Industry Scenarios

19.1 Banking and financial services

Regulated industries adopted OpenShift heavily because of its stricter default security posture (SCC), built-in audit logging, and the ability to run identical clusters on-premises (for data residency/regulatory requirements) and in the cloud (for burst capacity), using the same skills and tooling in both places. A common pattern: core banking systems stay on-premises on OpenShift for compliance, while customer-facing digital channels burst into a cloud-hosted OpenShift cluster for elastic scaling during peak load (e.g., month-end).

19.2 Telecommunications (telco) and edge computing

Telco networks increasingly run their own network functions (things that used to be dedicated hardware boxes) as containerized software on OpenShift, distributed across thousands of small sites (cell towers, regional offices). This is the single biggest real-world use case for ****Single Node OpenShift (SNO)**** — exactly the architecture you are building in your homelab — because each edge site is a small, single-server footprint that still needs the full OpenShift API and Operator ecosystem for centralized, consistent management across thousands of sites at once via Advanced Cluster Management.

19.3 Retail

Retailers run OpenShift both centrally (e-commerce platforms, inventory systems) and at the edge (in-store point-of-sale and inventory systems, sometimes with intermittent connectivity back to a central cluster), valuing the platform's consistency across wildly different hardware footprints, from large cloud clusters down to a small in-store server.

19.4 Healthcare

Similar regulatory drivers to banking (data residency, auditability), plus a growing overlap with AI: medical imaging analysis and clinical decision-support models increasingly run as containerized inference services on OpenShift AI, alongside the traditional patient records/administrative systems on the same underlying cluster technology.

19.5 Hybrid and multi-cloud strategy

Many large enterprises deliberately avoid being locked into one public cloud provider. Because OpenShift presents the same APIs whether it's running on AWS, Azure, GCP, bare metal, or vSphere, application teams write and deploy against one consistent platform, while infrastructure teams retain the freedom to choose or change the underlying cloud provider without rewriting applications — a major strategic reason large organisations adopt OpenShift over a cloud-native-only Kubernetes offering.

19.6 AI/ML workloads (directly relevant to you)

****OpenShift AI**** provides Jupyter notebook environments for data scientists, model training pipelines (often using KubeFlow-derived components running as Operators), and model serving (via KServe/ModelMesh, exposing a trained model as a scalable, versioned REST/gRPC endpoint) — all on the same cluster, same RBAC, same monitoring stack described throughout this book. This is the natural convergence point of your ML/AI background and your growing infrastructure/DevOps skill set.

19.7 A composite "day in the life" scenario, tying concepts together

A retail company's data science team trains a product-recommendation model in an OpenShift AI notebook (Chapter 10's Operator concept). A Tekton pipeline (Chapter 11) automatically packages the trained model into a serving container whenever a data scientist merges an update to a Git repository (GitOps, also Chapter 11). ArgoCD deploys the new model version to a staging Project first, where a readiness probe (Chapter 9) confirms it's healthy before a Route (Chapter 7) exposes it internally for QA testing. RBAC (Chapter 8) ensures only the ML team's ServiceAccount can push to this Project. Once approved, the same pipeline promotes it to production, where Prometheus and Grafana (Chapter 12) track inference latency, and an HPA (Chapter 18) scales the number of serving replicas up automatically during a Black Friday traffic spike — all without a human manually SSHing into a single server.

Chapter 20 — Common Troubleshooting Scenarios

20.1 Pod stuck in "Pending"

****Likely causes:**** insufficient CPU/RAM available on any node to satisfy the Pod's resource requests; a PVC the Pod needs hasn't bound to a PV yet; a node selector/affinity rule that no current node satisfies. ****First commands:**** ``oc describe pod `` (the Events section at the bottom almost always states the exact reason in plain English) and ``oc get nodes -o wide`` to check available capacity.

20.2 Pod stuck in "CrashLoopBackOff"

****Meaning:**** the container starts, then exits (crashes) repeatedly, and Kubernetes is backing off how often it retries. ****Likely causes:**** an application error at startup (bad config, missing environment variable), a failing liveness probe killing an otherwise-slow-starting app, or an SCC/permissions issue (Chapter 8) causing the process to fail when it tries to bind a privileged port or write to a restricted path as a non-root user. ****First commands:**** ``oc logs --previous`` (the `--previous`` flag shows the log from the crashed instance, not the fresh restart) and ``oc describe pod ``.

20.3 PVC stuck in "Pending"

****Likely causes:**** no StorageClass matches what was requested, no PV of sufficient size/access-mode exists for static provisioning, or the CSI driver/provisioner Pod itself is unhealthy. ****First commands:**** ``oc get storageclass``, ``oc describe pvc ``.

20.4 Node shows "NotReady"

****Likely causes:**** the kubelet has stopped reporting (node powered off, network partition, resource exhaustion causing the node to become unresponsive), or a Cluster Operator dependency (e.g., networking) is degraded. ****First commands:**** ``oc get nodes``, ``oc describe node ``, and — since this is infrastructure-level — checking the VM's actual power/console state directly in the ESXi/vSphere client.

20.5 Route/application unreachable from a browser, but the Pod is Running

****Likely causes, roughly in order of frequency:**** DNS not resolving the Route's hostname (very common in homelab setups — verify with ``nslookup`/`dig`` from the client machine, not just from inside the cluster); the Service's label selector not matching the Pod's labels (``oc get endpoints `` shows zero addresses if this is the problem); a missing or failing readiness probe keeping the Pod out of the Service's pool even though it "looks" Running; a firewall/NetworkPolicy blocking the path.

20.6 Build failing (S2I / BuildConfig)

****Likely causes:**** the source Git repository is unreachable (credentials, network), the detected builder image doesn't match the actual language/framework in the repo, or the build simply fails the same way it would fail on a developer's own machine (a genuine code/dependency error). ****First commands:**** ``oc logs -f bc`` during the build, ``oc get builds`` to see build history and status.

20.7 Cluster upgrade or ClusterOperator stuck "Degraded"

****Likely causes:**** insufficient resources for the upgrade's rolling process to complete, a certificate issue, or a genuine bug/incompatibility. ****First commands:**** ``oc get clusteroperators``, then ``oc describe clusteroperator`` on whichever one shows Degraded, which names the specific underlying failure.

Chapter 21 — Interview Preparation: Questions and Strong Answers

21.1 Conceptual questions

****Q:** What is the difference between Kubernetes and OpenShift?**

A: OpenShift is a Kubernetes distribution that adds a built-in image registry, a source-to-image build system, a web console, enterprise authentication (OAuth), stricter default security (SCC), and a curated Operator marketplace (OLM/OperatorHub) — all tested and supported together as one product, whereas plain Kubernetes provides only the orchestration core and leaves those pieces to be assembled separately.

****Q:** Explain the reconciliation loop.**

A: Every controller compares the desired state stored in etcd (via the API server) against the actual observed state of the cluster, and continuously takes action to close any gap. This declarative, self-healing model is why you declare "3 replicas should exist" rather than issuing one-time imperative commands.

****Q:** What's the difference between a Deployment and a DeploymentConfig?**

A: DeploymentConfig is OpenShift's original, now-legacy rollout object, predating Kubernetes' native Deployment; Deployment is the current, forward-looking, portable choice, and is what should be used for new work.

****Q:** How does an OpenShift Route differ from a Kubernetes Ingress?**

A: Route is OpenShift's native, richer object for exposing Services externally (with edge/passthrough/re-encrypt TLS options built in and no separate ingress controller to install); Ingress is the portable, Kubernetes-standard equivalent, which OpenShift also supports and automatically translates into a Route behind the scenes.

21.2 Scenario-based questions

****Q:** A Pod is stuck Pending — walk me through your troubleshooting.**

A: Start with ``oc describe pod`` and read the Events section, which almost always states the cause directly (insufficient resources, unbound PVC, unsatisfied node selector). Then check ``oc get nodes -o wide`` for available capacity, and ``oc get pvc`` if storage is involved.

****Q:** How would you roll back a bad deployment in production?**

A: ``oc rollout undo deployment/`` reverts to the previous ReplicaSet immediately; if a specific earlier revision is needed, ``oc rollout history deployment/`` lists revisions to target with ``--to-revision``.

****Q:** How would you restrict one team's application from being reachable by another team's Pods?**

A: A NetworkPolicy scoped to the relevant namespaces/labels, layered with RBAC to prevent the other team from even deploying into that Project in the first place.

21.3 More scenario and architecture questions

****Q:** What happens to a Pod's data when the Pod is deleted?**

A: Anything written to the container's own filesystem is lost immediately, because that filesystem is ephemeral and tied to the container instance. Only data written to a mounted PersistentVolume (via a PVC) survives, because a PV's lifecycle is independent of any one Pod.

****Q:** Why would you choose a StatefulSet over a Deployment?**

A: A Deployment's Pods are interchangeable and get random names/IPs; a StatefulSet gives each replica a stable, predictable identity (name and, with a headless Service, a stable DNS entry) and stable per-replica storage, which is required by clustered stateful systems like databases where each node needs to know its own identity and reconnect to its own data after a restart.

****Q:** Explain the difference between a Role and a ClusterRole with a concrete example.**

A: A Role granting "read Pods" only applies within the one namespace it's created in; the same permission as a ClusterRole applies cluster-wide. A common real pattern is defining permissions once as a ClusterRole (so the rule text isn't duplicated everywhere), then using a namespaced RoleBinding to grant that ClusterRole's permissions to a user in just one namespace at a time — reusing the rule, not the scope.

****Q:** A developer says their image works locally but fails to start on OpenShift with a permissions error. What do you check first?**

A: Whether the image assumes it can run as root or write to a fixed system path — check the SCC assigned to the Project (`oc get scc`), and check the image's Dockerfile for hardcoded UID/GID assumptions or writes to root-owned directories, since OpenShift's default `restricted-v2` SCC runs containers as an arbitrary non-root UID.

****Q:** How does OpenShift decide which node to schedule a Pod on?**

A: The scheduler evaluates every node's available CPU/memory against the Pod's `resources.requests`, applies any node affinity/anti-affinity rules and taints/tolerations the Pod specifies, and picks a node that satisfies all hard constraints — among the valid options, it favours spreading load evenly rather than always picking the same node.

****Q:** What's the difference between `oc delete pod` and `oc rollout restart deployment`?

A: Deleting a Pod directly triggers its owning ReplicaSet to create exactly one replacement immediately, with no version change — useful for "just restart this one instance." `oc rollout restart` recreates every Pod under a Deployment following the same rolling-update strategy as a normal deploy (respecting `maxUnavailable`/`maxSurge`), which is the safer choice when restarting an entire application deliberately, such as after a ConfigMap change that needs a restart to take effect.

****Q:** Why might an application need a headless Service instead of a normal one?

A: When client code needs to discover and connect to each individual backing Pod separately, rather than being load-balanced transparently across them — the classic example is a clustered database or a peer-to-peer system where each replica must know the individual addresses of its peers, not just "some healthy instance," which is exactly the DNS-per-Pod behaviour a headless Service provides.

****Q:** How would you debug a slow application on OpenShift where CPU and memory both look fine?

A: Check whether the container is CPU-throttled despite "fine" overall node CPU — a container hitting its own `resources.limits.cpu` ceiling gets throttled even on an otherwise idle node, which shows up in oc adm top pod` and cAdvisor/Prometheus throttling metrics, not in simple node-level CPU graphs; also check downstream dependencies (database, external API) and, if a service mesh (Chapter 25) is present, its per-hop latency data.`

21.4 A note on how to actually answer in an interview

Lead with the "why," not just the "what" — e.g., don't just say "a Service has a stable IP," say "a Service exists because Pods are disposable and get new IPs on every restart; something has to stay constant so other parts of the system can find them." This is exactly the pattern this book has used throughout, deliberately, so that explaining these ideas out loud should already feel natural to you.

Chapter 22 — Where to Go From Here

22.1 A realistic 90-day learning path building on this book

- bullet**Weeks 1-2:** Get CRC running (Chapter 16.1); work through Chapters 3, 4, 6, 7, 9 hands-on — deploy, expose, scale, and break your own test application on purpose so you experience the troubleshooting chapter firsthand.
- bullet**Weeks 3-5:** Build the Single Node OKD cluster on your ESXi host (Chapter 16.2); integrate your existing BIND DNS and RHEL 9 Ansible control node into the workflow (Chapter 17); practice Day-2 operations from Chapter 18 deliberately (upgrade, cordon/drain, a manual etcd backup).
- bullet**Weeks 6-8:** Install one real Operator from OperatorHub end-to-end (a database Operator is a good first choice); wire up a basic Tekton pipeline and/or ArgoCD for one of your own small applications, closing the loop from Git commit to running Pod automatically.
- bullet**Weeks 9-12:** If hardware allows, or via a free-tier cloud trial, attempt a genuine multi-node cluster to see high-availability behaviour firsthand (kill a control-plane node on purpose and watch the cluster keep functioning); begin exploring OpenShift AI concepts specifically, tying this back to your ML/AI background.

22.2 Certifications worth knowing about

Red Hat's certification track most relevant here is the ****EX280 (Red Hat Certified Specialist in OpenShift Administration)****, which sits under the broader ****RHCA (Red Hat Certified Architect)**** path and typically assumes ****RHCSA****-level Linux administration skill first — which lines up naturally with the RHCSA study material you are already working through.

22.3 Content creation angle for AI BlockMaker

Every chapter of this book is structured around one repeated teaching pattern — "what problem does this solve, then how it works, then a concrete example" — which maps directly onto short-form video scripts: each numbered section here (e.g., "Why Services exist," "What SCC actually restricts," "Pending vs CrashLoopBackOff") is close to a self-contained script for one video, broken into exactly the kind of digestible blocks your channel's tagline promises.

Chapter 23 — Annotated YAML: Reading and Writing Manifests Fluently

23.1 Why YAML, and why you must be able to read it without a console

Every object this book has described — Pod, Deployment, Service, Route, PVC, Role — is, underneath any `oc` command or web console click, just a structured text document. `oc new-app` and the console generate this text for you, but real production work, GitOps (Chapter 11), and every troubleshooting session eventually requires reading and editing this text directly. ****YAML (YAML Ain't Markup Language)**** was chosen because it is readable as plain English-like structure, using indentation instead of brackets.

23.2 The universal shape every object shares

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: demo
  labels:
    app: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: image-registry.openshift-image-registry.svc:5000/demo/myapp:latest
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
          readinessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
          livenessProbe:
            httpGet:
              path: /healthz
```

```
port: 8080
initialDelaySeconds: 15
```

Four fields appear on almost every object you will ever write, and understanding them is 80% of reading any manifest:

- bullet `apiVersion`` — which version of which API group this object belongs to (e.g., ``apps/v1`` for Deployments, ``v1`` for core objects like Pods and Services, ``route.openshift.io/v1`` for Routes).
- bullet `kind`` — the object type (Deployment, Service, Route, PersistentVolumeClaim, and so on).
- bullet `metadata`` — the object's name, namespace, and labels — how it is identified and found by other objects.
- bullet `spec`` — the desired state you are declaring; this is the part that differs completely between object kinds, and is what the relevant controller's reconciliation loop (Chapter 3) continuously works to satisfy.

Note also ``resources.requests`` and ``resources.limits``: **requests** are what the scheduler uses to decide which node has room for this Pod; **limits** are a hard ceiling the kubelet enforces at runtime (a container exceeding its memory limit is killed — "OOMKilled" — and exceeding its CPU limit is throttled, not killed). Forgetting to set these is one of the most common real-world causes of one noisy application starving its neighbours on a shared node.

23.3 A Service manifest, annotated

```
apiVersion: v1
kind: Service
metadata:
  name: myapp
  namespace: demo
spec:
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

The ``selector`` here is the exact mechanism from Chapter 3.5 — this Service finds its backing Pods purely by matching the label ``app: myapp``, nothing else. ``port`` is what other things inside the cluster connect to; ``targetPort`` is the actual port the container listens on — they do not have to match, and mismatching them by accident is a very common source of "Service exists but nothing responds" bugs.

23.4 A Route manifest, annotated

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: myapp
  namespace: demo
spec:
  host: myapp.apps.ocp4.lab.local
  to:
    kind: Service
```

```

    name: myapp
  port:
    targetPort: 80
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect

```

`to.name` must match a real Service name in the same namespace — this is how the Route (Chapter 7) hands off to the Service, which then hands off to a Pod. `insecureEdgeTerminationPolicy: Redirect` is a common, sensible default: it automatically redirects any plain HTTP request to HTTPS rather than serving insecurely.

23.5 A PersistentVolumeClaim manifest, annotated

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myapp-data
  namespace: demo
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: thin-csi

```

`storageClassName` tells the dynamic provisioner (Chapter 13) which backend/tier to use — on a vSphere-backed OpenShift install this is commonly a CSI StorageClass name like `thin-csi`, pointing at your ESXi datastore. Omitting `storageClassName` entirely uses the cluster's configured default StorageClass, which is fine for most learning purposes.

23.6 A ConfigMap and a Secret, side by side

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: myapp-config
data:
  LOG_LEVEL: "info"
  FEATURE_X_ENABLED: "true"
---
apiVersion: v1
kind: Secret
metadata:
  name: myapp-db-creds
type: Opaque
stringData:
  DB_USER: appuser
  DB_PASSWORD: change-me-in-real-life

```

Note the `---` separator: a single YAML file can contain multiple objects, applied together with one `oc apply -f`. Also note `stringData` (plain text, auto-encoded by the API server) versus the older `data` field (which requires you to pre-base64-encode values yourself) — `stringData` is the friendlier, less error-prone choice for hand-written Secrets.

23.7 Wiring a ConfigMap and Secret into a Pod

```
containers:
  - name: myapp
    image: ...
    envFrom:
      - configMapRef:
          name: myapp-config
      - secretRef:
          name: myapp-db-creds
```

`envFrom` injects every key in the ConfigMap/Secret as an environment variable automatically, without listing each one by name — the simplest way to wire configuration into a container, and the pattern you will use the most often in practice.

Chapter 24 — OpenShift Virtualization: Running VMs Alongside Containers

24.1 Why this chapter matters specifically to you

You already run workloads as full VMs on ESXi. **OpenShift Virtualization** (built on the open-source **KubeVirt** project) lets you run traditional virtual machines as first-class citizens inside an OpenShift cluster, managed with the same `oc`/console tooling, the same RBAC, the same networking and storage stack as containers — a VM becomes just another kind of workload the scheduler places, rather than a completely separate hypervisor layer to manage independently.

24.2 How it actually works under the hood

OpenShift Virtualization runs each VM inside a specially-constructed Pod (sometimes called a "virt-launcher" Pod), using **QEMU/KVM** (the same battle-tested Linux virtualization technology underlying many hypervisors) as the actual hardware emulation layer, with **libvirt** managing the QEMU process. From the outside, `oc get vm` and `oc get vmi` (VirtualMachine and VirtualMachineInstance objects — CRDs installed by the OpenShift Virtualization Operator, tying directly back to Chapter 10) show you VMs the same way `oc get pods` shows containers, and the VM's actual guest OS (Windows, RHEL, anything) runs unmodified inside.

24.3 Why an organisation would want this instead of just keeping VMware

- One platform, one team, one skill set** — instead of a separate virtualization team running vSphere and a separate platform team running OpenShift, both VMs and containers are operated through the same interface, the same monitoring stack (Chapter 12), and the same RBAC model (Chapter 8).
- Gradual modernization** — a legacy application that cannot easily be containerized yet (a licensing constraint, a vendor-locked appliance image, a Windows-only workload) can be imported as a VM onto the same cluster where its newer, containerized dependencies already run, letting teams modernize incrementally rather than needing a single big-bang migration.
- Migration tooling** — the **Migration Toolkit for Virtualization (MTV)** can import existing VMware VMs directly into OpenShift Virtualization, converting them into KubeVirt-managed VMs — directly relevant if you ever wanted to experimentally migrate one of your own existing ESXi VMs as a learning exercise.

24.4 A minimal VM manifest, for orientation

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: legacy-app-vm
spec:
  running: true
  template:
```

```
spec:
  domain:
    cpu:
      cores: 2
    memory:
      guest: 4Gi
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
  volumes:
    - name: rootdisk
      persistentVolumeClaim:
        claimName: legacy-app-disk
```

Notice the shape is deliberately familiar — ``metadata``, ``spec``, resource requests — following the exact same conventions from Chapter 23, because OpenShift Virtualization was explicitly designed to feel native rather than bolted-on.

Chapter 25 — Service Mesh: When Applications Need to Talk to Each Other Intelligently

25.1 The problem that appears once you have many microservices

A Service (Chapter 7) solves "how do I reach a stable address for a group of Pods." It does not solve a harder set of problems that appear once an application is split into dozens of interdependent microservices: How do I encrypt all traffic between services automatically, without every developer implementing TLS themselves? How do I retry a failed call to a flaky downstream service safely? How do I gradually shift 5% of traffic to a new version to test it safely (canary deployment)? How do I see a complete map of which service calls which, and how long each call takes? A **service mesh** answers all of these questions at the infrastructure layer, without application code changes.

25.2 How a service mesh works: the sidecar pattern

OpenShift Service Mesh (based on the open-source **Istio** project, plus **Envoy** proxies and **Kiali** for visualisation) works by automatically injecting a small proxy container (Envoy) into every application Pod as a second container — the "sidecar" pattern briefly mentioned in Chapter 3.4. Every bit of network traffic in and out of the application container is transparently intercepted by this sidecar, which is where mesh features (mutual TLS, retries, traffic splitting, detailed metrics) actually get implemented, completely invisibly to the application code itself.

25.3 Concrete capabilities and why they matter

- **Mutual TLS (mTLS) everywhere, automatically** — every service-to-service call is encrypted and both sides cryptographically verify each other's identity, without a single line of application code, directly strengthening the security posture described in Chapter 8.
- **Traffic splitting / canary releases** — route 95% of traffic to the current version and 5% to a new version, watch its error rate and latency in Kiali/Grafana, and shift the percentage up gradually — a far safer rollout strategy than an all-at-once Deployment rolling update for high-stakes services.
- **Circuit breaking and retries** — automatically stop sending traffic to a failing downstream service for a short period rather than piling more requests onto something already struggling, and automatically retry transient failures — resilience patterns that would otherwise need to be hand-coded, inconsistently, in every single service.
- **Distributed tracing integration** — feeds directly into the tracing pillar from Chapter 12.3, since the mesh sees literally every request crossing every service boundary.

25.4 When you actually need one

A service mesh adds real operational complexity and resource overhead (an extra container per Pod, extra configuration surface). It earns its cost once an application genuinely has many interdependent

services with real cross-service reliability and security requirements — not for a single monolithic application, and often not for a small homelab learning cluster. Knowing **when not to** reach for a service mesh is itself a mark of real architectural judgment and a very reasonable thing to say in an interview.

Chapter 26 — Multi-Tenancy, Governance, and Cost Awareness

26.1 What "multi-tenancy" means in this context

A single OpenShift cluster is very often shared by multiple teams, departments, or even external customers ("tenants"), each of whom should be able to work independently without stepping on each other or seeing each other's data. Multi-tenancy is not one feature — it is the combined effect of several mechanisms already covered separately in this book, applied together deliberately: Projects (Chapter 4) for isolation boundaries, RBAC (Chapter 8) for who can do what, NetworkPolicy (Chapter 7) for network isolation, and SCC (Chapter 8) for workload-level restriction.

26.2 ResourceQuota and LimitRange — stopping one tenant from starving everyone else

ResourceQuota — a per-Project cap on total resource consumption (e.g., "this Project may request at most 8 CPU and 16 GB RAM across all its Pods combined, and at most 20 Pods").

LimitRange — sets default and maximum resource requests/limits for individual Pods/containers within a Project, so a developer who forgets to specify `resources`` (Chapter 23.2) still gets a sane default rather than an unbounded container.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
  namespace: team-a
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    pods: "20"
```

26.3 Why this matters even in a homelab of one

Even on your own single-node cluster, setting a small ResourceQuota on a test Project is genuinely useful practice: it teaches you to observe exactly what happens when a Deployment tries to exceed its quota (`oc describe`` will show a clear rejection reason), which is a very common real support ticket in any organisation running shared clusters.

26.4 Cost awareness on a homelab specifically

Unlike a cloud cluster, your homelab has no per-hour billing to watch — but the equivalent constraint is your fixed 32 GB of RAM and 320 GB of disk (Chapter 15). The same discipline that controls cloud cost in a real organisation (rightsizing `resources.requests``, scaling down unused Deployments to zero replicas rather than leaving them idle, cleaning up abandoned PVCs) directly controls whether your homelab stays usable or grinds to a halt from resource exhaustion — the underlying skill transfers perfectly even though the "cost" is measured in gigabytes rather than dollars.

26.5 Chargeback and showback in real organisations

Larger real-world deployments often integrate cost-visibility tooling (Red Hat's **Cost Management** service, or the open-source **OpenCost** project) that attributes actual cluster resource consumption back to individual Projects/teams/cost-centers, turning the abstract ResourceQuota numbers above into an actual dollar figure a department can be held accountable for — worth knowing exists, even though it's out of scope for a homelab.

Chapter 27 — How OpenShift Compares to Other Platforms

27.1 Why this comparison matters

Job descriptions rarely say only "OpenShift" — they mix in EKS, AKS, GKE, Rancher, and plain "Kubernetes," and interviewers frequently ask you to place OpenShift among these. This chapter gives you a grounded, fair comparison rather than marketing language, so you can speak about all of them credibly.

27.2 Managed cloud Kubernetes: EKS, AKS, GKE

Amazon EKS (Elastic Kubernetes Service), **Azure AKS (Azure Kubernetes Service)**, and **Google GKE (Google Kubernetes Engine)** are each a cloud provider's managed control plane for plain, mostly-vanilla Kubernetes — the provider runs and patches the control plane for you, but you are still responsible for assembling most of what OpenShift bundles in (ingress controller, registry, CI/CD, monitoring, developer console) from separate add-ons, typically installed via Helm charts.

Trade-off: these are lighter-weight and often cheaper to start with if you're already committed to one specific cloud, but you take on more integration and consistency work yourself, and you lose the "same experience everywhere" benefit described in Chapter 19.5 the moment you need a second environment (on-prem, or a different cloud).

27.3 Rancher

Rancher (from SUSE) is a multi-cluster management layer that can sit on top of Kubernetes clusters from many sources (including EKS/AKS/GKE, on-prem clusters, or its own lightweight **K3s** distribution) and provides a unified console and policy layer across all of them. It overlaps conceptually with Red Hat's Advanced Cluster Management (Chapter 5.6) more than it overlaps with OpenShift itself — Rancher is often compared against "OpenShift plus ACM" as a combined offering, rather than against bare OpenShift.

27.4 Plain/vanilla Kubernetes (self-assembled)

Running upstream Kubernetes yourself (via `kubeadm`, or a lightweight distribution like **K3s** or **MicroK8s**) gives you maximum flexibility and the smallest possible footprint — genuinely relevant for very constrained environments — at the cost of assembling and maintaining every non-core piece yourself, and losing the coordinated, tested upgrade story the Cluster Version Operator (Chapter 5.3) provides. K3s in particular is worth knowing about for extremely small edge devices (Raspberry Pi-class hardware) where even Single Node OpenShift's footprint is too heavy.

27.5 A grounded summary table

Platform	Control plane managed by	Built-in registry/CI/console	Typical use case
OpenShift (OCP/OKD)	You (on-prem) or Red Hat (cloud-managed variants)	Yes, all included	Enterprise, regulated, hybrid, or multi-cloud environments wanting one consistent platform
EKS / AKS / GKE	Cloud provider	No — add-ons required	Teams already committed to one cloud, wanting less operational overhead for the control plane specifically
Rancher	You, or a mix	Multi-cluster management, not a distribution itself	Managing many heterogeneous clusters from one pane of glass
kubeadm / K3s / MicroK8s	You	No	Learning the rawest form of Kubernetes, or extremely resource-constrained edge devices

27.6 The honest answer to "which one is best"

There is no universally best choice — the right answer always depends on the organisation's existing cloud commitments, regulatory constraints, in-house operational skill, and how many different environments (on-prem, multiple clouds, edge) need to look and behave consistently. What is true, and worth saying directly in an interview, is that the **concepts** in this book — Pods, Services, Deployments, RBAC, the reconciliation loop — are identical across every option in the table above, because they are all Kubernetes underneath. What differs is how much of the "everything else" (Chapters 4-12 of this book) is included, tested, and supported for you versus left for your team to assemble.

Chapter 28 — A Complete End-to-End Worked Project

28.1 The goal of this chapter

Every previous chapter explained one concept at a time. This chapter deliberately combines almost everything into a single, realistic project you can actually build on your homelab cluster (CRC or your Single Node OKD build from Chapter 16) — a small "notes" web application backed by a database, built from source, exposed externally, monitored, and eventually wired into a simple pipeline.

28.2 The scenario

You will deploy a small two-tier application: a web front end (built from your own Git repository using S2I) and a PostgreSQL database (installed via an Operator from OperatorHub), connected by a Secret holding the database credentials, exposed externally by a Route, protected by a NetworkPolicy that only allows the front end to reach the database, and scaled with a HorizontalPodAutoscaler.

28.3 Step 1 — create an isolated Project with a quota

```
oc new-project notes-app
oc apply -f - << EOF
apiVersion: v1
kind: ResourceQuota
metadata:
  name: notes-quota
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 4Gi
    pods: "10"
EOF
```

This immediately puts Chapter 26's governance concepts into practice, even for a single-user homelab project — good habit-building for real multi-tenant environments later.

28.4 Step 2 — install PostgreSQL via an Operator

From the console's OperatorHub (Chapter 10.3), install a PostgreSQL Operator (e.g., Crunchy Data's or another OperatorHub-listed option) into the `notes-app` Project, then create a small Custom Resource requesting a single-instance database:

```
apiVersion: postgres-operator.example.com/v1
kind: PostgresCluster
metadata:
  name: notes-db
spec:
  instances: 1
  storage:
    size: 5Gi
```

Watch the Operator's reconciliation loop (Chapter 10.1) do the actual work — a Pod, a PVC (Chapter 13), and a Secret containing the generated credentials all appear automatically within a minute or two, without you writing any of those objects by hand.

28.5 Step 3 — build and deploy the front end from source

```
oc new-app https://github.com/example/notes-frontend --name=notes-web
oc set env deployment/notes-web --from=secret/notes-db-pguser-notes-db
oc expose svc/notes-web
```

`oc set env --from=secret/...` wires the Operator-generated database credentials directly into the front end's environment variables — connecting Chapter 9's build workflow to Chapter 10's Operator output without ever hand-writing connection strings.

28.6 Step 4 — restrict network access with a NetworkPolicy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-web-only
spec:
  podSelector:
    matchLabels:
      postgres-operator.example.com/cluster: notes-db
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: notes-web
```

Now only Pods labelled `app: notes-web` can reach the database — everything else in the Project, and anything in any other Project, is denied by default, directly applying Chapter 7.7's concept.

28.7 Step 5 — add a HorizontalPodAutoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: notes-web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: notes-web
  minReplicas: 1
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

This directly implements Chapter 18.3's scaling concept: replicas grow automatically toward 4 if average CPU utilization across existing replicas passes 70%, and shrink back down when load drops.

28.8 Step 9 — verify the whole system end to end

```
oc get route notes-web
oc get hpa notes-web-hpa
oc get networkpolicy
oc describe pod -l app=notes-web
```

Open the Route's URL in a browser, submit a note, and confirm it persists (proving the database connection and PVC are genuinely working) — then intentionally delete the front-end Pod (`oc delete pod -l app=notes-web`) and watch a replacement appear automatically, restoring service without you touching anything, which is Chapter 3.5's reconciliation loop made concrete and visible on your own screen.

28.9 What this project teaches, chapter by chapter

Concept exercised	Chapter reference
Projects and quotas	4, 26
Operators, CRDs, OLM	10
S2I builds	9
Secrets wiring	8, 23
Storage via Operator-managed PVC	13
Routes	7
NetworkPolicy	7.7
Autoscaling	18.3
Reconciliation loop, self-healing	3.5

Chapter 29 — Frequently Asked Beginner Questions

29.1 "Do I need to know Docker before learning OpenShift?"

Basic container concepts (Chapter 2) are essential — you must understand what an image and a container are — but you do not need deep Docker-specific expertise, since OpenShift's own build system (S2I) can take you from source code straight to a running container without you ever hand-writing a Dockerfile. That said, knowing how to read a Dockerfile is still valuable, since many real-world images you'll deploy were built from one by someone else.

29.2 "Is OpenShift the same as Kubernetes, just renamed?"

No — see Chapter 4 in full, but the short answer is that OpenShift includes Kubernetes entirely unmodified underneath (every `kubectl`-equivalent object and API works exactly the same way), and adds a substantial, opinionated layer of additional tooling, defaults, and support on top.

29.3 "Why does my container fail with a permissions error that never happened on Docker Desktop?"

Almost always the SCC restriction from Chapter 4.2 and 8.4 — OpenShift runs your container as a random non-root UID by default, while Docker Desktop typically runs everything as root unless told otherwise. Fix the image to be non-root-friendly (write only to directories the container's own arbitrary UID can access, avoid binding to ports below 1024), rather than reaching for a more permissive SCC as a first resort.

29.4 "What is the actual difference between a Project and a Namespace?"

A Project is a Namespace plus OpenShift-specific metadata and, usually, self-service creation rules for regular (non-admin) users — every Project is backed by a real Namespace object, and `oc get namespace` will show it too.

29.5 "Can I just install Docker Desktop's Kubernetes and call it close enough to OpenShift?"

For learning the raw Kubernetes objects from Chapter 3, yes, it's a reasonable substitute. For learning anything OpenShift-specific (Routes, SCC, S2I, the console, Operators/OLM, the Cluster Version Operator) — no, none of that exists there, which is exactly why Chapters 16's CRC and SNO paths matter: they give you the real thing, not an approximation.

29.6 "Is my 32 GB homelab actually 'real world' experience, or just a toy?"

Genuinely real. A Single Node OpenShift cluster (Chapter 16.2) is a supported, production-grade topology used at real telco and retail edge sites (Chapter 19.2) — the only thing your homelab lacks compared to a large enterprise deployment is scale (fewer nodes, smaller quotas), not correctness or authenticity of the underlying architecture. Everything from Chapters 3 through 13 behaves identically whether the cluster has 1 node or 500.

29.7 "How long does it realistically take to become 'good' at this?"

Chapter 22's 90-day plan is a genuine, achievable path to solid working competence — comfortable deploying, troubleshooting, and explaining the architecture. Deeper expertise (the kind reflected in the EX280 certification from Chapter 22.2, or true Day-2 mastery at enterprise scale) typically takes 6-12 months of continued hands-on work beyond that, which is normal and not a sign you're behind.

29.8 "What should I actually build first, this week?"

Chapter 16.1 (CRC) followed immediately by Chapter 28's worked project, end to end, on that CRC cluster. Doing the full worked project once, even imperfectly, teaches more than reading every remaining chapter in isolation — this is deliberate advice, not filler: hands-on repetition is what actually converts this book's explanations into skills you can use under interview or incident pressure.

Chapter 30 — Preparing ESXi Itself, Click by Click

30.1 Why this chapter exists separately from Chapter 16

Chapter 16 walked through the OpenShift/OKD installer's perspective. This chapter walks through the ESXi/vSphere client side specifically — the settings that are easy to get wrong on a standalone ESXi host (no vCenter) and that silently cause an otherwise-correct OpenShift install to fail or run poorly.

30.2 Confirm virtualization prerequisites on the ESXi host itself

Since RHCOS (and CRC's internal VM, if you run CRC inside a VM on ESXi rather than on bare metal) itself expects certain CPU features, and because you may eventually run OpenShift Virtualization (Chapter 24) which needs nested virtualization to test inside a VM:

- bullet Confirm your physical host's CPU supports Intel VT-x or AMD-V, and that it is enabled in the physical machine's BIOS/UEFI settings (not just in ESXi) — this is required for ESXi itself to run VMs at all, so if you're already running VMs today this is already satisfied.
- bullet If you intend to nest virtualization (running CRC's internal VM inside an ESXi VM, or testing OpenShift Virtualization's KVM layer inside a VM), expose virtualization to the guest: edit the VM's settings, under **CPU**, enable **"Expose hardware assisted virtualization to the guest OS."** Without this, nested VMs inside your OpenShift VM will fail to start or fall back to very slow emulation.

30.3 Creating the VM for a Single Node OKD build

In the ESXi web client (`https://ui``):

- 1 **Virtual Machines → Create/Register VM → Create a new virtual machine.**
- 2 Name it clearly, e.g., `ocp4-sno``. Guest OS family: **Linux**; Guest OS version: **Red Hat Enterprise Linux 9 (64-bit)** — RHCOS is close enough to RHEL 9 for ESXi's guest customisations to work correctly, even though it isn't literally RHEL.
- 3 **Select storage** — choose your datastore; leave the disk as **Thin provisioned** to conserve your 320 GB SSD across other VMs, accepting the small ongoing risk described in Chapter 16.3.
- 4 **Customize hardware:**
 - bullet **CPU:** as many vCPU as you can spare (Chapter 15 recommends 6-8); leave "Hardware virtualization" exposed if you plan to nest anything later.
 - bullet **Memory:** 16-20 GB, and tick **"Reserve all guest memory (All locked)"** if you want to guarantee this VM never gets ESXi-level memory pressure/swapping at the cost of that RAM being unavailable to any other VM while this one is powered on — a reasonable choice for a primary learning cluster you want to behave predictably.
 - bullet **New Hard Disk:** 120-150 GB, Thin provisioned, VMware Paravirtual or LSI Logic SAS controller (Paravirtual generally performs better if the guest supports it).

- bullet**New Network:** attach to the port group/virtual switch you planned in Chapter 15.3; set the adapter type to **VMXNET3** (the paravirtualized, higher-performance NIC — the default "E1000" emulated NIC works but is measurably slower).
 - bullet**New CD/DVD Drive:** set to **Datastore ISO file**, and browse to the live ISO you generated/downloaded as part of the OKD/OCP install process (Chapter 16.2); tick **Connect at power on**.
 - bullet**Boot Options:** confirm **Firmware** matches what your chosen OpenShift/OKD version expects (recent versions commonly expect **EFI**, not legacy BIOS) — a firmware mismatch here is a subtle, easy-to-miss cause of an installer that simply won't boot.
- 1 **Finish**, then power on and immediately open the VM's console to watch the live ISO boot and begin the Ignition-driven install process described in Chapter 16.2.

30.4 A pre-flight checklist worth running every time

- bullet[] DNS records (``api``, ``api-int``, ``*.apps``) resolve correctly from a machine other than the VM itself.
- bullet[] The ESXi datastore has at least the full 150 GB genuinely free (not just "thin provisioning says it's fine") before starting.
- bullet[] No other memory-heavy VM is powered on competing for the RAM this VM needs, unless you've reserved memory as above.
- bullet[] The correct ISO is attached and set to connect at power-on.
- bullet[] Boot firmware (EFI vs BIOS) matches what the installer expects.
- bullet[] You have the ``install-config.yaml`` and generated Ignition files backed up somewhere outside the VM itself, since you cannot regenerate them identically without the same pull secret and SSH key inputs.

30.5 What "healthy" looks like immediately after first boot

The console will show a standard Linux boot sequence, then RHCOS-specific Ignition messages applying configuration, then (for SNO) an automatic reboot partway through as the bootstrap-in-place process hands off from a temporary bootstrap state to the final single-node control plane. This can reasonably take 30-60+ minutes on modest hardware — resist the urge to assume it has hung during the middle portion where the console may show little visible output; use ``openshift-install wait-for install-complete`` from your separate installer machine (Chapter 16.2, step 6) as the authoritative progress signal rather than the console screen alone.

Chapter 31 — Object Kind Quick Reference

31.1 Why a table like this earns its place

Across this book you have met dozens of object `kind` values and their `apiVersion`. This single table is meant to be the one place you look things up once you're past the conceptual explanation and just need the exact string to type.

Kind	apiVersion	Purpose	Chapter
Pod	v1	Smallest deployable unit; one or more co-located containers	3
ReplicaSet	apps/v1	Keeps N identical Pod copies running	3
Deployment	apps/v1	Manages ReplicaSets; rolling updates and rollbacks	3, 9
DeploymentConfig	apps.openshift.io/v1	Legacy OpenShift-native rollout object	4
Service	v1	Stable virtual IP/DNS in front of a set of Pods	3, 7
Route	route.openshift.io/v1	Exposes a Service externally with a hostname and TLS	4, 7
Ingress	networking.k8s.io/v1	Portable, Kubernetes-native external exposure object	3, 7
NetworkPolicy	networking.k8s.io/v1	Restricts Pod-to-Pod network traffic	7
Namespace	v1	A virtual sub-cluster boundary	3
Project	project.openshift.io/v1	OpenShift's Namespace-plus-metadata equivalent	4
ConfigMap	v1	Non-sensitive configuration data	3, 23

Kind	apiVersion	Purpose	Chapter
Secret	v1	Sensitive configuration data	3, 23
PersistentVolume	v1	Real storage capacity made available to the cluster	13
PersistentVolumeClaim	v1	A request for storage by an application	13, 23
StorageClass	storage.k8s.io/v1	Template describing how to dynamically create PVs	13
BuildConfig	build.openshift.io/v1	Describes how to build an image from source	4, 9
Build	build.openshift.io/v1	One executed run of a BuildConfig	4
ImageStream	image.openshift.io/v1	Tracks image tag versions and triggers redeployments	4
Role	rbac.authorization.k8s.io/v1	Namespaced permission set	8
ClusterRole	rbac.authorization.k8s.io/v1	Cluster-wide permission set	8
RoleBinding	rbac.authorization.k8s.io/v1	Grants a Role to a user/group in one namespace	8
ClusterRoleBinding	rbac.authorization.k8s.io/v1	Grants a ClusterRole cluster-wide	8
ServiceAccount	v1	Identity for a process/Pod	8
SecurityContextConstraints	security.openshift.io/v1	Governs what a Pod is allowed to do at the OS level	4, 8
ResourceQuota	v1	Caps total resource use within a Project	26
LimitRange	v1	Default/maximum resource settings per Pod/container	26

Kind	apiVersion	Purpose	Chapter
HorizontalPodAutoscaler	autoscaling/v2	Automatically scales replicas based on metrics	18, 28
CustomResourceDefinition	apiextensions.k8s.io/v1	Adds a new object type to the API	10
Subscription	operators.coreos.com/v1alpha1	Tells OLM to install/track an Operator	10
ClusterServiceVersion	operators.coreos.com/v1alpha1	Describes a specific installed Operator version	10
Pipeline / PipelineRun	tekton.dev/v1	Tekton CI/CD pipeline definition and a single run of it	11
Application	argoproj.io/v1alpha1	ArgoCD's definition of a GitOps-managed application	11
VirtualMachine	kubevirt.io/v1	A VM managed by OpenShift Virtualization	24
ClusterOperator	config.openshift.io/v1	Health/version status of a core cluster component	5
MachineSet	machine.openshift.io/v1beta1	Declaratively manages a group of worker node machines	18

Chapter 32 — How Your RHCSA Study Connects Directly to OpenShift

32.1 Why this bridge matters

You are working through RHCSA chapters on storage, LVM, the boot process, and recovery at the same time as this book. These are not two separate tracks that happen to overlap in schedule — RHCSA-level Linux skill is the literal foundation every OpenShift node runs on, and understanding that connection makes both subjects easier to retain.

32.2 Storage and LVM → PersistentVolumes

The RHCSA skill of creating physical volumes, volume groups, and logical volumes with `pvcreate`/`vgcreate`/`lvcreate` is exactly what, at a lower level, provides the raw block storage that a CSI driver (Chapter 13) formats and hands out as PersistentVolumes. When you troubleshoot a PVC stuck Pending (Chapter 20.3), the underlying cause is sometimes not a Kubernetes-layer problem at all, but a genuine out-of-space or misconfigured-storage problem at exactly the LVM/filesystem layer your RHCSA study covers directly.

32.3 Boot process → understanding node startup and RHCOS

RHCSA's boot-process chapters (GRUB, initramfs, systemd targets) map directly onto understanding what actually happens during the RHCOS boot sequence described in Chapter 30.5 — RHCOS is still fundamentally a Linux boot process underneath Ignition's extra first-boot configuration step, and recognising a genuinely stuck boot (versus a slow-but-healthy one) draws on exactly this knowledge.

32.4 Recovery techniques → cluster-level disaster recovery instincts

RHCSA teaches single-machine recovery: booting into rescue/emergency mode, repairing a broken bootloader, resetting a lost root password. OpenShift's Day-2 disaster recovery (Chapter 18.4) is the same underlying discipline applied at cluster scale — "how do I get back to a known-good state when something is broken" — just with etcd snapshots and static Pod manifests standing in for a single machine's filesystem backup.

32.5 SELinux and firewalld → SCC and NetworkPolicy

SELinux's philosophy (deny by default, explicitly allow) is the same philosophy SCC (Chapter 8.4) applies to container permissions, and firewalld's zone-based traffic rules are the same conceptual shape as NetworkPolicy (Chapter 7.7) applied inside the cluster instead of at the host firewall. If RHCSA's SELinux troubleshooting (`ausearch`, `sealert`, context labels) already feels familiar, you already have the right mental model for debugging SCC-related permission denials — the specific commands differ, but the reasoning pattern ("something is denied by default, find out exactly what and why, grant the minimum needed") is identical.

32.6 The honest takeaway

You are not learning two unrelated skill sets in parallel. You are learning one operating system deeply (RHCSA) and one orchestration layer built directly on top of it (OpenShift) — nearly every OpenShift Day-2 problem eventually traces back to a Linux-level fact your RHCSA study is teaching you to recognise directly.

Chapter 33 — Self-Test: Check Your Understanding

33.1 How to use this chapter

Answer each question in your own words, out loud, before checking it against the chapter referenced — this rehearsal is what actually converts reading into recall under pressure, whether that pressure is an interview or a 2 a.m. incident.

33.2 Foundational questions

- 1 In one sentence, why does Kubernetes exist, and in one sentence, why does OpenShift exist on top of Kubernetes? *(Chapters 2.4, 1.2)*
- 2 What is the difference between a container and a virtual machine, at the kernel level? *(Chapter 2.1)*
- 3 Name the four core control-plane components and one job each. *(Chapter 3.2)*
- 4 What problem does a Service solve that a Pod's own IP address cannot? *(Chapter 3.4)*
- 5 Describe the reconciliation loop in your own words, using an example that isn't from this book. *(Chapter 3.5)*

33.3 OpenShift-specific questions

- 1 What does the default SCC prevent a container from doing, and why does that default exist? *(Chapter 4.2, 8.4)*
- 2 Name the three Route TLS termination types and one scenario where each is the right choice. *(Chapter 7.5)*
- 3 What is the difference between a BuildConfig, a Build, and an ImageStream? *(Chapter 4.5)*
- 4 Why would a team choose Deployment over the older DeploymentConfig for new work? *(Chapter 4.6)*
- 5 What does OLM actually automate, beyond just "installing software"? *(Chapter 10.3)*

33.4 Homelab and infrastructure questions

- 1 Given your 32 GB / 320 GB ESXi host, why is a full 3-master production topology not realistic, and what's the recommended alternative? *(Chapter 15.1-15.2)*
- 2 What three DNS records must exist and resolve correctly before you boot an OKD/OCP installer VM? *(Chapter 15.3, 16.2)*
- 3 Why is IPI's vSphere path generally not available on a standalone ESXi host without vCenter, and what do you use instead? *(Chapter 14.1)*
- 4 What ESXi-level VM setting is required if you want to nest virtualization inside an OpenShift VM? *(Chapter 30.2)*

33.5 Troubleshooting scenarios

- 1 A Pod is Pending. What are your first two commands, and what are you looking for in the output? *(Chapter 20.1)*
- 2 A Pod is CrashLoopBackOff, but ``oc logs `` shows nothing useful. What flag do you add, and why? *(Chapter 20.2)*
- 3 A Route resolves in DNS but returns a connection error. Besides DNS, what two other things would you check, in order? *(Chapter 20.5)*
- 4 A PVC is stuck Pending. Which three objects would you inspect, and in what order? *(Chapter 20.3)*

33.6 Answer-writing exercise

Pick any three questions above and write a two-to-three sentence answer for each as if explaining it in a job interview, leading with "why," not "what," exactly as Chapter 21.3 describes. This is, deliberately, the single most useful thirty minutes you can spend after finishing this book.

Appendix — Command Cheat Sheet

A.1 Cluster and login

```
oc login -u <user> -p <password> <api-url>
oc whoami
oc project <name>
oc new-project <name>
oc get clusteroperators
oc get nodes -o wide
```

A.2 Deploying and inspecting applications

```
oc new-app <git-url> --name=<name>
oc get pods
oc get deployment
oc get svc
oc get route
oc expose svc/<name>
oc logs -f <pod>
oc logs -f bc/<name>
oc describe pod <pod>
oc rsh <pod>
```

A.3 Scaling and rollout

```
oc scale deployment/<name> --replicas=<n>
oc rollout status deployment/<name>
oc rollout history deployment/<name>
oc rollout undo deployment/<name>
```

A.4 Storage

```
oc get pvc
oc get pv
oc get storageclass
oc describe pvc <name>
```

A.5 Security and access

```
oc get rolebindings
oc adm policy add-role-to-user <role> <user> -n <project>
oc get scc
oc describe scc restricted-v2
```

A.6 Day-2 / node operations

```
oc adm cordon <node>
oc adm drain <node> --ignore-daemonsets
oc adm uncordon <node>
oc adm upgrade
oc adm upgrade --to-latest
```

A.7 Troubleshooting

```
oc get events --sort-by=.lastTimestamp
oc describe node <node>
oc get endpoints <service>
oc logs <pod> --previous
oc get builds
```

Closing note: this cheat sheet is meant to be used, not memorised. Keep it open in a second VS Code tab while you work through Chapters 16-18 on your own cluster — that repetition is what actually builds the muscle memory an interview or a real incident will demand from you.